

```
In [3]: linbias = 30  
[linX, linY, lincoefs] = datasets.make_regression(n_samples=100, n_  
n_informative=1,  
coef=True, random  
  
# outliers  
outbias = 10  
[outX, outY, outcoefs] = datasets.make_regression(n_samples=10, n_f  
n_informative=1,  
coef=True, random
```

```

# combine
outlinX = r_[linX, 2*outX/2]
outlinY = r_[linY, outY]

# fit using ordinary least squares
ols = linear_model.LinearRegression()
ols.fit(outlinX, outlinY)

# fit using ridge regression
alpha = 100
rr = linear_model.Ridge(alpha=alpha)
rr.fit(outlinX, outlinY)

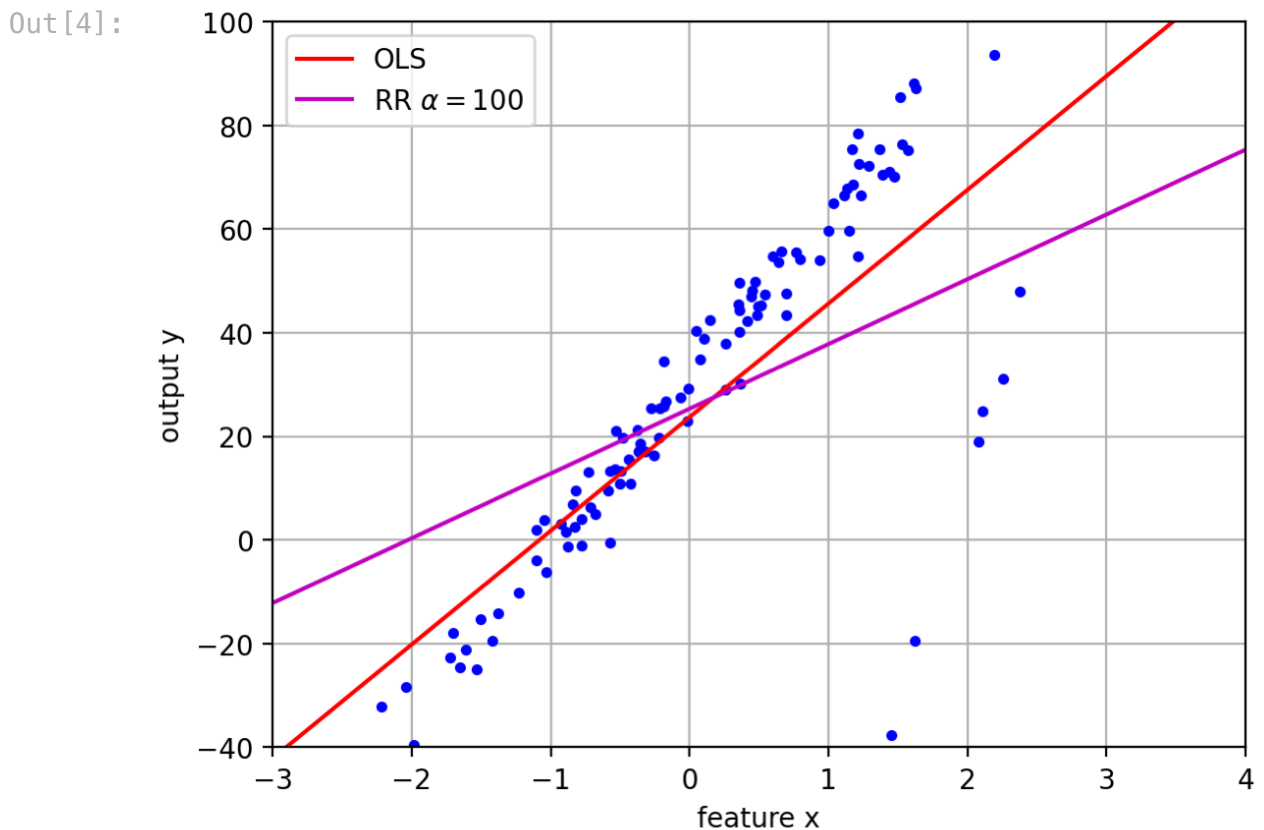
oabox = [-3, 4, -40, 100]

xr = linspace(oabox[0], oabox[1], 100).reshape(100,1)
ypred = ols.predict(xr)
ypredrr = rr.predict(xr)

outfig = plt.figure()
plt.plot(outlinX, outlinY, 'b.')
plt.plot(xr, ypred, 'r-', label="OLS")
plt.plot(xr, ypredrr, 'm-', label='RR  $\alpha=100$ ')
plt.xlabel('feature x'); plt.ylabel('output y')
plt.grid(True); plt.axis(oabox)
plt.legend(loc='upper left')
plt.close()

```

In [4]: outfig



RANSAC

- **RAN**dom **SA**mples **C**onsensus
 - attempt to robustly fit a regression model in the presence of corrupted data (outliers).
 - works with any regression model.
- **Idea:**
 - split the data into inliers (good data) and outliers (bad data).
 - learn the model only from the inliers

Random sampling

- Repeat many times with random subset of points (usually just enough to fit the model)
 - fit a model to the subset.
 - classify all data as inlier or outlier by calculating the residuals (prediction errors) and comparing to a threshold. The set of inliers is called the *consensus set*.
 - save the model with the highest number of inliers.
- Use the largest consensus set to learn the final model.

```
In [5]: random.seed(1002)
ransacfig = plt.figure(figsize=(12,3))
for i in range(3):
    inds = [random.choice(range(len(outlinY))) for j in range(2)]

    ols = linear_model.LinearRegression()
    ols.fit(outlinX[inds], outlinY[inds])

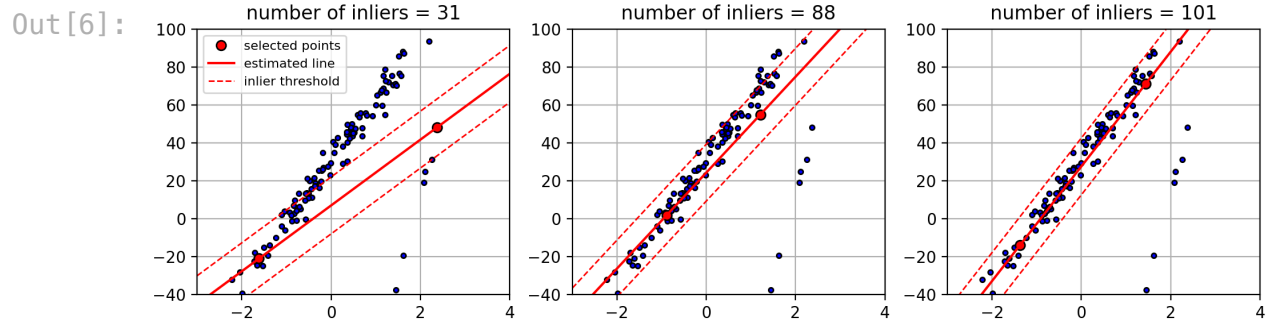
    ypred = ols.predict(xr)
    T = 15

    # calculate number of inliers
    ytpred = ols.predict(outlinX)
    numin = sum(abs(ytpred-outlinY)<=T)

    plt.subplot(1,3,i+1)
    plt.plot(outlinX, outlinY, 'b.', markeredgecolor='k')
    plt.plot(outlinX[inds], outlinY[inds], 'ro', label='selected po
    plt.plot(xr, ypred, 'r-', label='estimated line')
    plt.plot(xr, ypred+T, 'r--', linewidth=1, label='inlier thresho
    plt.plot(xr, ypred-T, 'r--', linewidth=1)
    plt.grid(True); plt.axis(oaxbox)
    plt.title("number of inliers = " + str(numin))
    if i==0:
        plt.legend(fontsize=8, loc='upper left')
```

```
plt.close()
```

```
In [6]: ransacfig
```



RANSAC

- More iterations increases the probability of finding the correct function.
 - higher probability to select a subset of points contains all inliers.
- Threshold typically set as the median absolute deviation of y .

```
In [7]: def plot_regr_trans_1d(model, axbox, X, Y, feattrans=None, numx=100):
        xr = linspace(axbox[0], axbox[1], numx)

        # predict the function
        if feattrans != None:
            xrf = feattrans(xr[:,newaxis])
        else:
            xrf = xr[:,newaxis]

        if model.__class__.__name__ == "GaussianProcessRegressor":
            Ypred, Ystd = model.predict(xrf, return_std=True)
            hasstd = True
        else:
            Ypred = model.predict(xrf)
            hasstd = False

        if hasstd:
            plt.fill_between(xr, Ypred - 2*Ystd, Ypred + 2*Ystd,
                             alpha=0.2, color='k')

        plt.plot(X, Y, 'b.')
        plt.plot(xr, Ypred, 'r-')
        plt.axis(axbox); plt.grid(True)
```

```
In [8]: # use RANSAC model (defaults to linear regression)
        rlin = linear_model.RANSACRegressor(random_state=1234)
        rlin.fit(outlinX, outlinY)

        inlier_mask = rlin.inlier_mask_
        outlier_mask = logical_not(inlier_mask)
```

```
In [9]: rfig = plt.figure()
```

```

plot_regr_trans_1d(rlin, oabox, outlinX, outlinY)
plt.plot(outlinX[inlier_mask], outlinY[inlier_mask], 'b.', label='i')
plt.plot(outlinX[outlier_mask], outlinY[outlier_mask], 'co', label='o')
leg = plt.legend(fontsize=8, loc='upper left')
plt.title('estimated (w,b) = (%0.4g,%0.4g)\ntrue (w,b) = (%0.4g,%0.4g)'
          (rlin.estimate.coef_, rlin.estimate.intercept_, linco
plt.close()

```

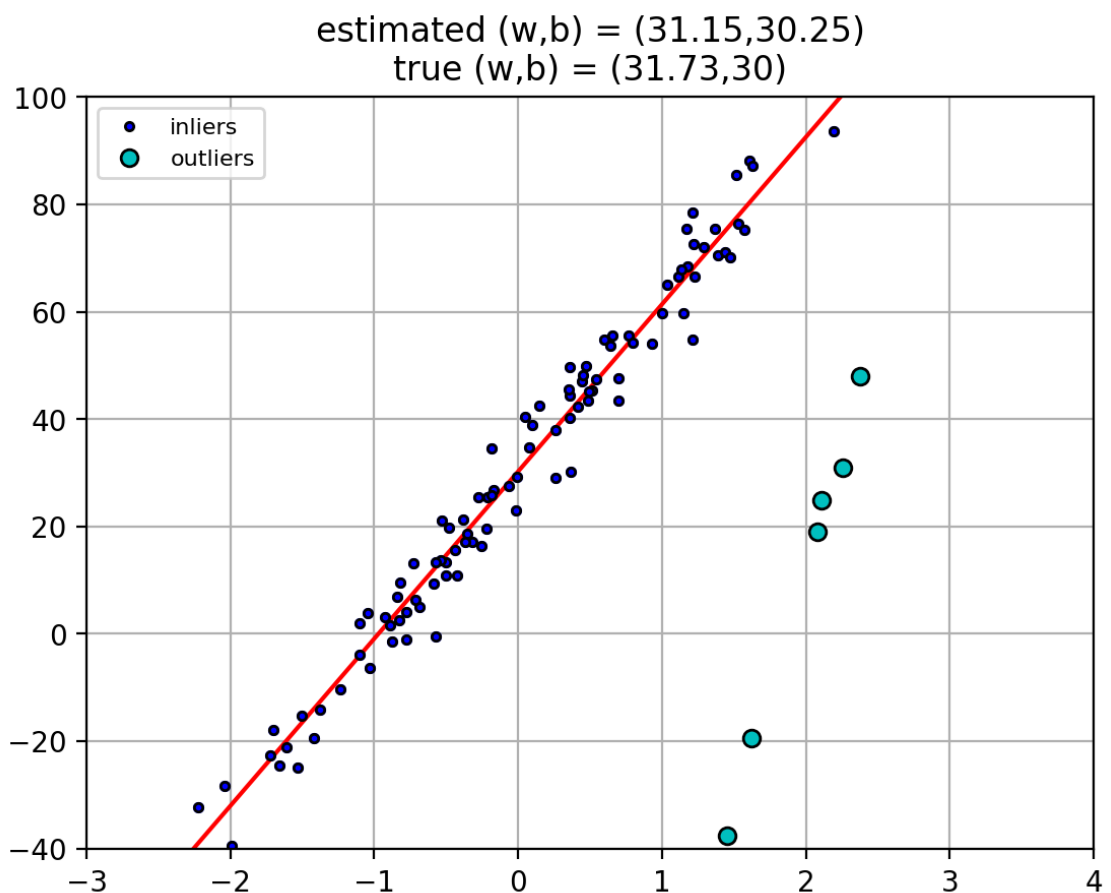
```

/var/folders/v0/s4761j_n3z7_f651dfw62m5w0000gn/T/ipykernel_8114/3701
789536.py:6: DeprecationWarning: Conversion of an array with ndim >
0 to a scalar is deprecated, and will error in future. Ensure you ex
tract a single element from your array before performing this operat
ion. (Deprecated NumPy 1.25.)
plt.title('estimated (w,b) = (%0.4g,%0.4g)\ntrue (w,b) = (%0.4g,%
0.4g)' %

```

In [10]: rfig

Out[10]:



Non-linear regression

- So far we have only considered linear regression: $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$
- Similar to classification, we can do non-linear regression by forming a feature vector of $\mathbf{x}$ and then performing linear regression on the feature vector.

Polynomial regression

- p-th order Polynomial function
 - $f(x) = w_0 + w_1x + w_2x^2 + w_3x^3 + \dots + w_px^p$
- Collect the terms into a vector

$$\text{▪ } f(x) = [w_0 \quad w_1 \quad w_2 \quad \dots \quad w_p] * \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^p \end{bmatrix} = \mathbf{w}^T \phi(x)$$

$$\text{▪ weight vector } \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_p \end{bmatrix}; \text{ polynomial feature vector:}$$

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^p \end{bmatrix}$$

- Now it's a linear function, so we can use the same linear regression!

Example

- 1st to 6th order polynomials

```
In [11]: # example data
polyX = random.normal(size=200)
polyY = sin(polyX) + 0.1*random.normal(size=200)
polyX = polyX[:,newaxis]

plin = {}
for d in [1,2,3,4,5,6]:
    # extract polynomial features with degree d
    polyfeats = preprocessing.PolynomialFeatures(degree=d)
    polyXf = polyfeats.fit_transform(polyX)

    # fit the parameters
    plin[d] = linear_model.LinearRegression()
    plin[d].fit(polyXf, polyY)
```

```
In [12]: polyfig = plt.figure(figsize=(9,4))
naxbox = [-4, 4, -2, 2]
```

```

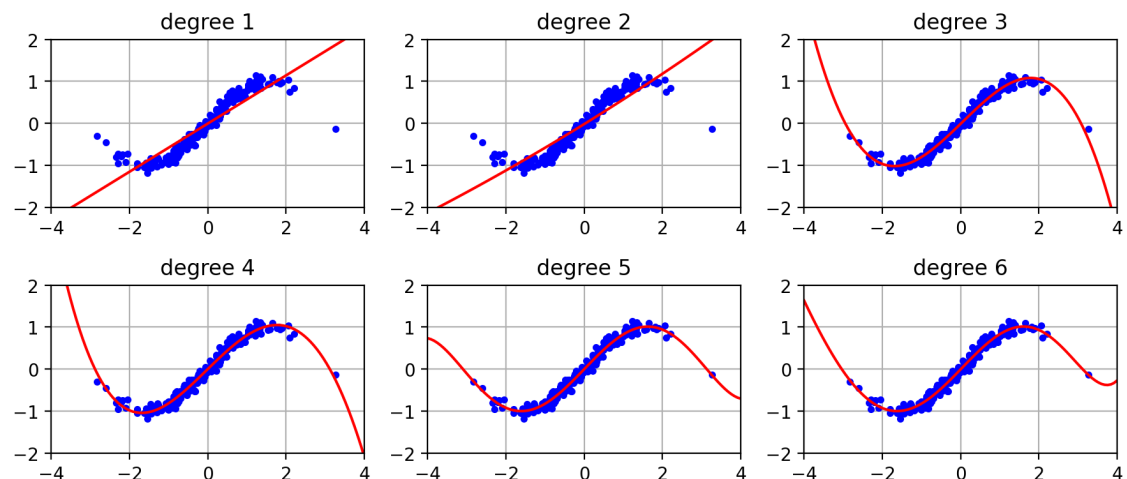
for d in [1,2,3,4,5,6]:
    # extract polynomial features with degree d
    polyfeats = preprocessing.PolynomialFeatures(degree=d)
    polyXf = polyfeats.fit_transform(polyX)

    # make plot
    plt.subplot(2,3,d)
    plot_regr_trans_1d(plin[d], naxbox, polyX, polyY, polyfeats.trans
    plt.title("degree " + str(d))
plt.tight_layout()
plt.close()

```

In [13]: polyfig

Out[13]:



Example: Housing data

- Using AgeSold feature
- Increasing polynomial degree d will decrease MSE of training data
 - more complicated model always fits data better
 - (but it could overfit)

```

In [14]: from sklearn.datasets import fetch_openml
def load_housing(mode=1):
    housing = fetch_openml(name="house_prices", as_frame=True)

    # 2-dim data for illustration
    if mode == 1:
        housingY = array(housing['target'])
        housingX = array(housing['data'][["GrLivArea", "YrSold", "Y

    # get age when sold
    housingX[:,1] -= housingX[:,2]
    housingX = housingX[:,0:2]

    ss = 4 # subsample for illustration
    housingY = housingY[::ss] / 1000
    housingX = housingX[::ss] / [[100,1]]

```

```

housingYname = 'Home Price ($1000s)'
housingXname = ('Living Area (100 sf)', 'Age when sold (Yrs'
housingXdesc = ('Living Area (100 sf)', 'Age when sold (Yrs'
housingaxbox = ([4.50,37,0,600], [0,140,0,600])

elif (mode==2):
    # data for illustrating feature selection

    housingY = array(housing['target'])
    housingX = array(housing['data']["GrLivArea", "TotRmsAbvGr",
                                     "GarageArea", "TotalBsmtS",
                                     "BedroomAbvGr", "FullBath",
                                     "LotArea", "LowQualFinSF",
                                     "OverallQual", "OverallCo",
                                     "YrSold", "YearBuilt",])

    # get age when sold
    housingX[:, -2] -= housingX[:, -1]
    housingX = housingX[:, 0:-1]

    ss = 4 # subsample for illustration
    housingY = housingY[::ss] / 1000
    housingX = housingX[::ss]

    housingYname = 'Home Price ($1000s)'
    housingXname = ["LiveA", "Rooms",
                    "GaraA", "Bsmt", "Fire",
                    "Bdrms", "Bath",
                    "LotA", "LowQA",
                    "Qual", "Cond",
                    "Age"]
    housingXdesc = ["Living Area (sqf)", "Number of Rooms", "Ga",
                    "Basement Area (sqf)", "Number of Fireplaces",
                    "Number of Bathrooms", "Lot Area (sqf)", "Ar",
                    "Overall Quality of Materials", "Overall Con"]
    housingaxbox = ()

    print("target: ", housingY.shape, "; data: ", housingX.shape)
    return housingX, housingY, housingXname, housingYname, housinga

```

```

In [15]: housingX, housingY, housingXname, housingYname, housingaxbox, housi
# get 2nd feature only
housingX = housingX[:,1][:,newaxis]
housingaxbox = housingaxbox[1]
housingXname = housingXname[1]

```

target: (365,) ; data: (365, 2)

```

In [16]: polyfeats = {}
        plin = {}
        MSE = {}
        for d in [1,2,3,4,5,6]:
            # extract polynomial features with degree d
            polyfeats[d] = preprocessing.PolynomialFeatures(degree=d)
            housingXf = polyfeats[d].fit_transform(housingX)

```

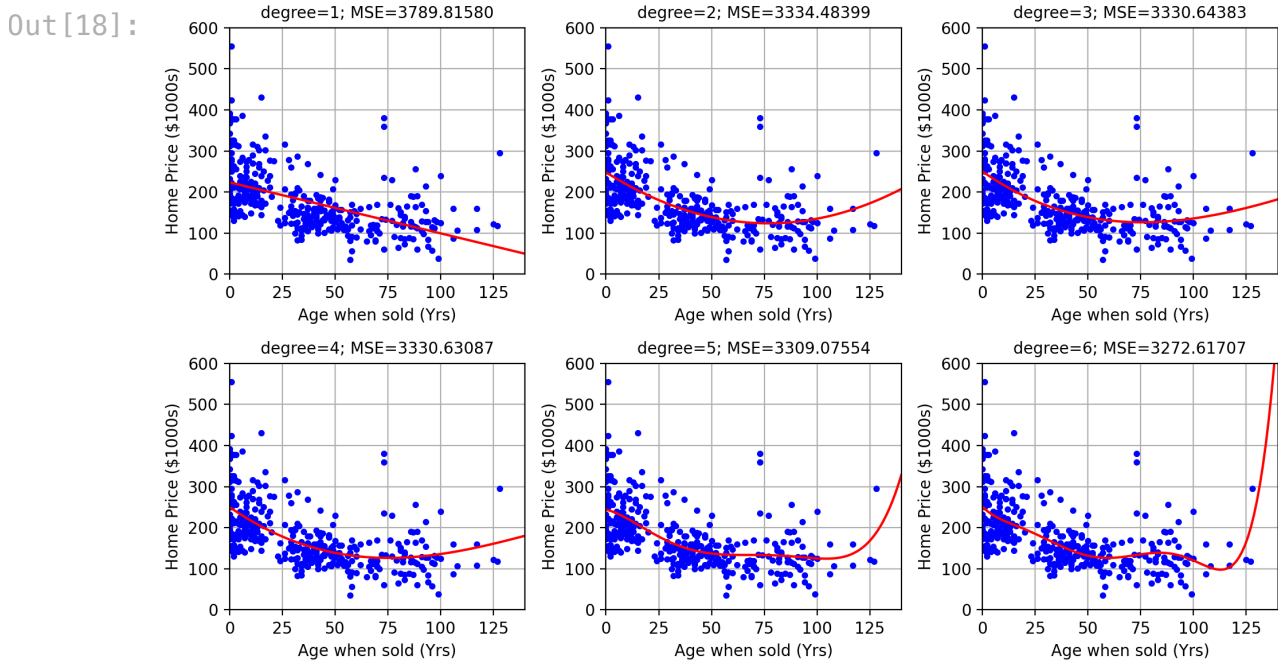


```
# fit the parameters
plin[d] = linear_model.LinearRegression()
plin[d].fit(housingXf, housingY)

# calculate mean-square error on training set
MSE[d] = metrics.mean_squared_error(housingY, plin[d].predict(h
```

```
In [17]: pfig = plt.figure(figsize=(10,6))
for d in [1,2,3,4,5,6]:
    # make plot
    plt.subplot(2,3,d)
    plot_regr_trans_1d(plin[d], housingaxbox, housingX, housingY, p
    plt.ylabel(housingYname)
    plt.title('degree={}; MSE={:.5f}'.format(d, MSE[d]), fontsize=1
    plt.xlabel(housingXname)
plt.tight_layout()
plt.close()
```

```
In [18]: pfig
```



Select degree using Cross-Validation

- Minimizing the MSE on the training set will overfit
 - More complex function always has lower MSE on training set
- Use cross-validation to select the proper model
 - the parameters we want to change are in feature transformation step
 - Use `pipeline` to merge all steps into one object for easier cross-validation
- `pipeline` object
 - pass an array of stages

- each entry is a tuple with the stage name and transformer (implements `.fit`, `.transform`)
- the last entry should be a model (implements `.fit`)

```
In [19]: # make the pipeline
polylin = pipeline.Pipeline([
    ('polyfeats', preprocessing.PolynomialFeatures(degree=1)),
    ('linreg', linear_model.LinearRegression())
])
```

```
In [20]: # set the parameters for grid search
# the parameters in each stage are named: <stage>__<parameter>
paramgrid = {
    "polyfeats__degree": array([1, 2, 3, 4, 5, 6]),
}

# do the cross-validation search - use -MSE as the score for maxim
plincv = model_selection.GridSearchCV(polylin, paramgrid, cv=5, n_j
    scoring='neg_mean_squared_err
plincv.fit(housingX, housingY)

print(plincv.best_params_)

{'polyfeats__degree': np.int64(2)}
```

```
In [21]: def extract_grid_scores(modelcv, paramgrid):
    "extract CV scores from GridSearchCV and put into a matrix"
    # get parameters
    pkeys = list(paramgrid.keys())

    # initialize empty score matrix
    scoresize = [len(paramgrid[p]) for p in pkeys]
    avgscores = zeros(scoresize)

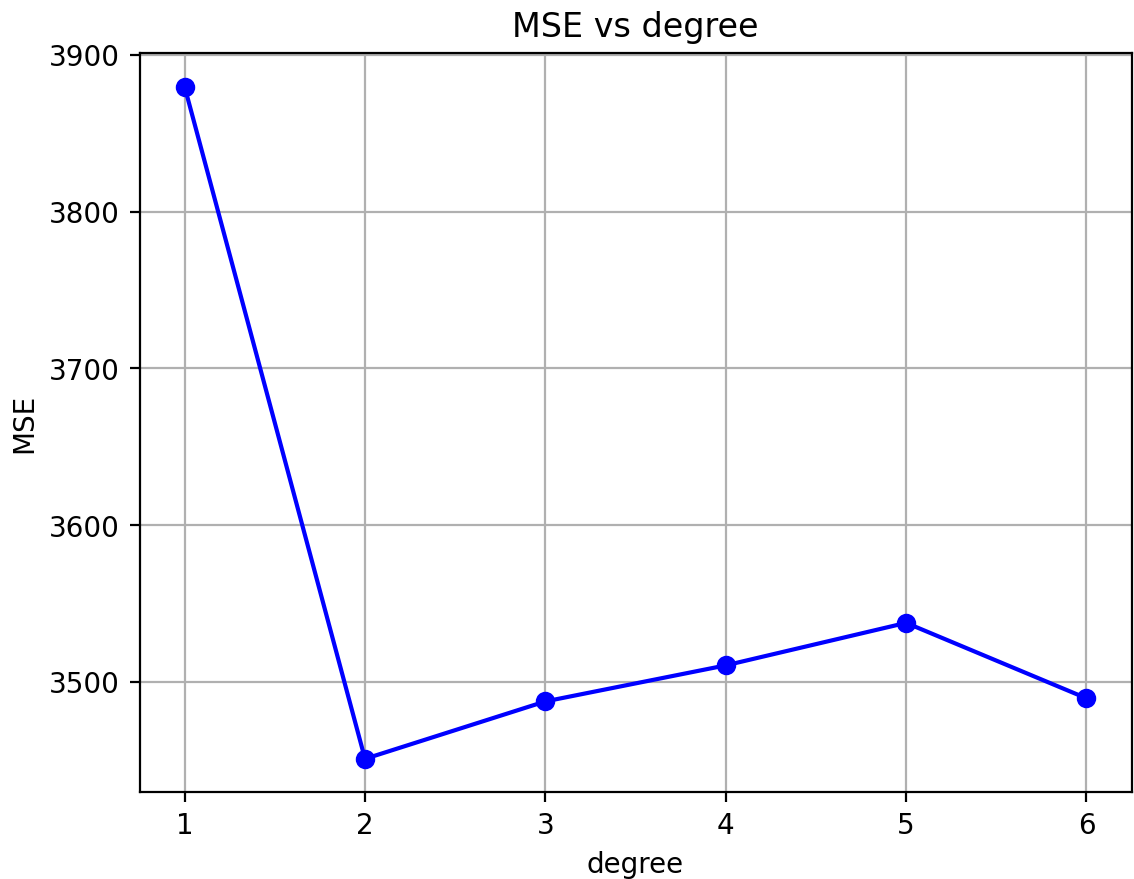
    # fill in the matrix with run for each parameter set
    for rm, rp in zip(modelcv.cv_results_['mean_test_score'], modelc
        # get the index into each of the parameter lists
        myind = [where(rp[p] == paramgrid[p]) for p in pkeys]
        avgscores[tuple(myind)] = rm # put the score

    # location of best score
    bestind = [where(modelcv.best_params_[p] == paramgrid[p]) for p

    return avgscores, pkeys, bestind
```

```
In [22]: avgscores, pnames, bestind = extract_grid_scores(plincv, paramgrid)
plt.figure()
plt.plot(paramgrid['polyfeats__degree'], -avgscores, 'bo-')
plt.xlabel('degree'); plt.ylabel('MSE'); plt.grid(True)
plt.title('MSE vs degree')
```

```
Out[22]: Text(0.5, 1.0, 'MSE vs degree')
```



Polynomial features: 2D Example

- 2D feature vectors:
 - $\mathbf{x} = [x_1 \quad x_2]^T$
- degree 2 polynomial transformation:

$$\phi(\mathbf{x}) = [x_1^2 \quad x_1x_2 \quad x_2^2]^T$$

- degree 3 polynomial transformation:

$$\phi(\mathbf{x}) = [x_1^3 \quad x_1^2x_2 \quad x_1x_2^2 \quad x_2^3]^T$$

```
In [23]: def plot_regr_trans_2d(model, axbox, X, Y, feattrans=None):
# maek grid
xr = [linspace(axbox[0],axbox[1],100),
      linspace(axbox[2],axbox[3],100)]
xgrid0, xgrid1 = meshgrid(xr[0], xr[1])
allpts = c_[xgrid0.ravel(), xgrid1.ravel()]
if feattrans != None:
    allptsf = feattrans(allpts)
else:
    allptsf = allpts

if model.__class__.__name__ == "GaussianProcessRegressor":
    tmpY, tmpstd = model.predict(allptsf, return_std=True)
```

```

ypred = tmpY.reshape(xgrid0.shape)
ystd = tmpstd.reshape(xgrid0.shape)
hasstd = True

tmpYX, tmpstdX = model.predict(X, return_std=True)
else:
    ypred = model.predict(allptsf).reshape(xgrid0.shape)
    hasstd = False

Yr = Y.max()-Y.min()
vmin = Y.min()-0.1*Yr
vmax = Y.max()+0.1*Yr

if not hasstd:
    plt.imshow(ypred, origin='lower', extent=axbox, aspect='auto',
               vmin=vmin, vmax=vmax, cmap='jet')
    plt.scatter(X[:,0], X[:,1], c=Y, vmin=vmin, vmax=vmax, cmap=

    plt.grid(True); plt.axis(axbox)
else:
    plt.subplot(1,2,1)
    plt.imshow(ypred, origin='lower', extent=axbox, aspect='auto',
               vmin=vmin, vmax=vmax, cmap='jet')
    plt.scatter(X[:,0], X[:,1], c=Y, vmin=vmin, vmax=vmax, cmap=
    plt.grid(True); plt.axis(axbox)

Yr = ystd.max()-ystd.min()
vmin = ystd.min()-0.1*Yr
vmax = ystd.max()+0.1*Yr

plt.subplot(1,2,2)
plt.imshow(ystd, origin='lower', extent=axbox, aspect='auto',
           vmin=vmin, vmax=vmax, cmap='jet')
plt.scatter(X[:,0], X[:,1], c=tmpstdX, vmin=vmin, vmax=vmax)
plt.grid(True); plt.axis(axbox)

```

In [24]: housingX, housingY, housingXname, housingYname, housingaxbox, housi
housingaxbox2 = housingaxbox[0][0:2]+housingaxbox[1][0:2]

target: (365,) ; data: (365, 2)

```

In [25]: plin = {}
polyfeats = {}
MSE = {}
for i,d in enumerate([2,3]):
    # get polynomial features
    polyfeats[d] = preprocessing.PolynomialFeatures(degree=d)
    housingXf = polyfeats[d].fit_transform(housingX)

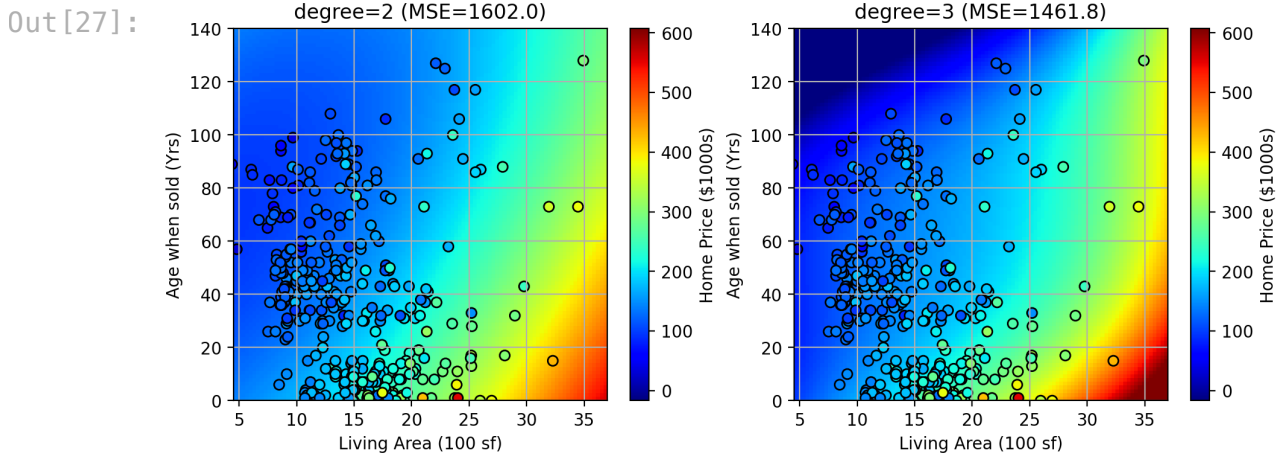
    # learn with both dimensions
    plin[d] = linear_model.LinearRegression()
    plin[d].fit(housingXf, housingY)

    # calculate MSE
    MSE[d] = metrics.mean_squared_error(housingY, plin[d].predict(h

```

```
In [26]: pfig = plt.figure(figsize=(11,4))
for i,d in enumerate([2,3]):
    # make figure
    plt.subplot(1,2,i+1)
    plot_regr_trans_2d(plin[d], housingaxbox2, housingX, housingY,
        cbar = plt.colorbar())
    cbar.set_label(housingYname)
    plt.xlabel(housingXname[0]); plt.ylabel(housingXname[1])
    plt.title('degree=%d (MSE=%.1f)' % (d, MSE[d]))
plt.close()
```

In [27]: pfig



Kernel Ridge Regression

- Apply *kernel trick* to ridge regression
 - turn linear regression into non-linear regression
 - use kernel $k(x, x')$

- Closed form solution:

- for an input point $\mathbf{x}_*$,

- prediction: $y_* = \mathbf{k}_*^T (\mathbf{K} + \alpha I)^{-1} \mathbf{y}$

$$\mathbf{K} = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{bmatrix} \text{ is the kernel matrix}$$

$(N \times N)$

- $\mathbf{k}_* = [k(\mathbf{x}_1, \mathbf{x}_*), \dots, k(\mathbf{x}_N, \mathbf{x}_*)]^T$ is vector containing the kernel values between $\mathbf{x}_*$ and all training points $\mathbf{x}_i$.

Example: Polynomial Kernel

- Note: it's the same as using polynomial features and linear regression!
 - Using the kernel, we don't need to explicitly calculate the polynomial

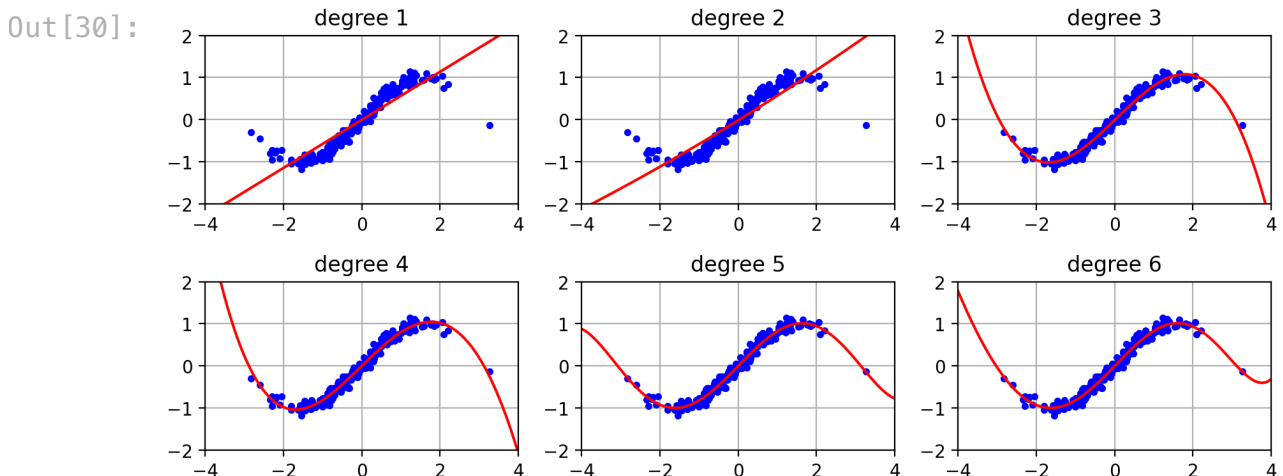
features.

- But, we do need to calculate the kernel function between all pairs of training points.

```
In [28]: krr = {}  
for d in [1,2,3,4,5,6]:  
    # fit the parameters  
    krr[d] = kernel_ridge.KernelRidge(alpha=1, kernel='poly', degree  
    krr[d].fit(polyX, polyY)
```

```
In [29]: krrfig = plt.figure(figsize=(9,4))  
  
for d in [1,2,3,4,5,6]:  
    # plot the function  
    plt.subplot(2,3,d)  
    plot_regr_trans_1d(krr[d], naxbox, polyX, polyY)  
    plt.title("degree " + str(d))  
plt.tight_layout()  
plt.close()
```

```
In [30]: krrfig
```



Example: RBF kernel

- gamma controls the smoothness
 - small gamma will estimate a smooth function
 - large gamma will estimate a wiggly function

```
In [31]: krr = {}  
for i,g in enumerate(logspace(-3,2,6)):  
    # fit the parameters  
    krr[i] = kernel_ridge.KernelRidge(alpha=1, kernel='rbf', gamma=  
    krr[i].fit(polyX, polyY)
```

```
In [32]: krrfig = plt.figure(figsize=(9,4))
```

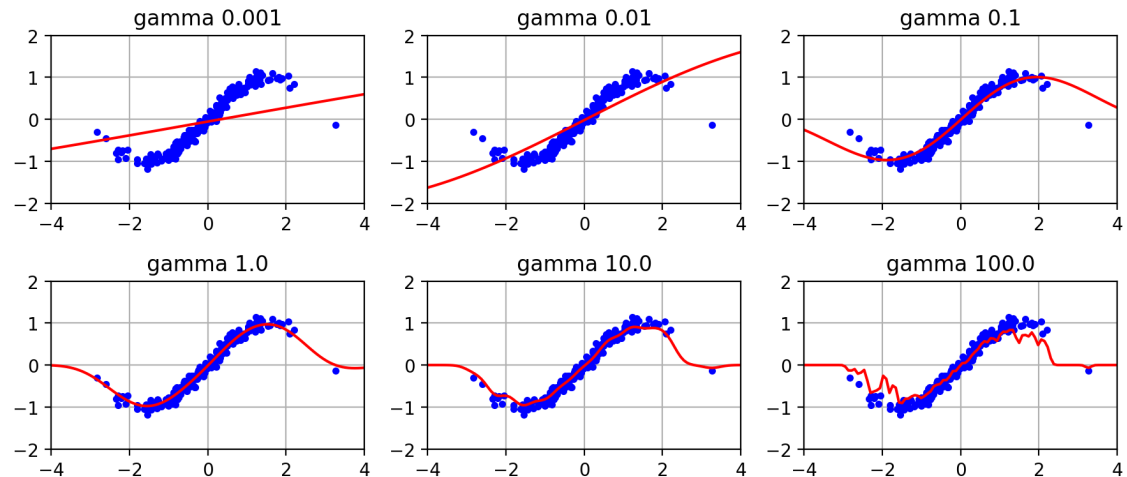
```

for i,g in enumerate(logspace(-3,2,6)):
    # plot the function
    plt.subplot(2,3,i+1)
    plot_regr_trans_1d(krr[i], naxbox, polyX, polyY)
    plt.title("gamma " + str(g))
plt.tight_layout()
plt.close()

```

In [33]: krrfig

Out[33]:



Housing Data: Cross-validation

- RBF kernel
 - cross-validation to select α and γ .

```

In [34]: # parameters for cross-validation
paramgrid = {'alpha': logspace(-3,3,10),
             'gamma': logspace(-3,3,10)}

# do cross-validation
krrcv = model_selection.GridSearchCV(
    kernel_ridge.KernelRidge(kernel='rbf'), # estimator
    paramgrid,                             # parameters to try
    scoring='neg_mean_squared_error',       # score function
    cv=5,                                  # number of folds
    n_jobs=-1, verbose=True)
krrcv.fit(housingX, housingY)

print(krrcv.best_score_)
print(krrcv.best_params_)

```

Fitting 5 folds for each of 100 candidates, totalling 500 fits

-1616.5500709702105

```
{'alpha': np.float64(0.004641588833612777), 'gamma': np.float64(0.001)}
```

In [35]: (avgscores, pnames, bestind) = extract_grid_scores(krrcv, paramgrid)

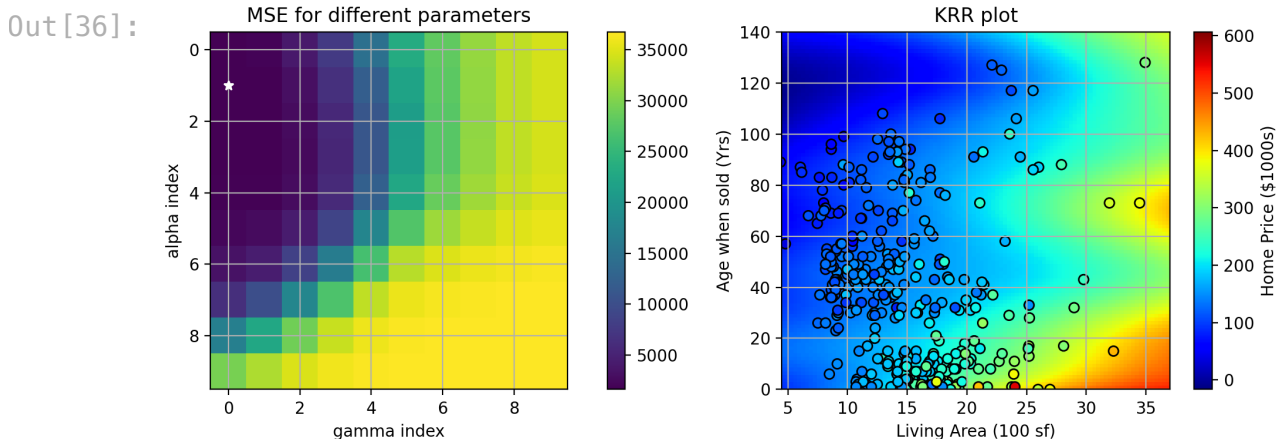
```

kfig = plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(-avgcores, interpolation='nearest')
plt.plot(bestind[1], bestind[0], '*w')
plt.ylabel(pnames[0] + ' index'); plt.xlabel(pnames[1] + ' index')
plt.grid(True)
plt.title('MSE for different parameters')
plt.colorbar(); plt.axis('image')

plt.subplot(1,2,2)
plot_regr_trans_2d(krrcv, housingaxbox2, housingX, housingY)
cbar = plt.colorbar()
cbar.set_label(housingYname)
plt.xlabel(housingXname[0])
plt.ylabel(housingXname[1])
plt.title('KRR plot')
plt.tight_layout()
plt.close()

```

In [36]: kfig



Gaussian Process Regression

- *Gaussian Process* is an infinite collection of r.v.s where any finite subset of r.v.s is joint Gaussian distributed.
 - infinite collection of values -> function
 - GP is prior distribution over **functions**.
- Denoted as: $f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$
 - function value: $f(\mathbf{x})$ is a distribution of f at location $\mathbf{x}$.
 - mean function: $m(\mathbf{x})$ is the mean function. Usually $m(\mathbf{x}) = c$, a constant.
 - covariance function: $\text{cov}(f(\mathbf{x}), f(\mathbf{x}')) = k(\mathbf{x}, \mathbf{x}')$
 - covariance of function values depends on inputs through the kernel k .
- For any $(x_1, \dots, x_N)$, the distribution of function values is:

$$f_1, \dots, f_N | \mathbf{x}_N, \dots, \mathbf{x}_N \sim \mathcal{N}(\mathbf{o}, \mathbf{K})$$

- $\mathbf{K}$ is the kernel matrix for points $\{\mathbf{x}_N, \dots, \mathbf{x}_N\}$

Examples of GP priors

- the kernel defines the types of functions that are regressed

```
In [37]: from sklearn.gaussian_process.kernels import DotProduct, WhiteKernel

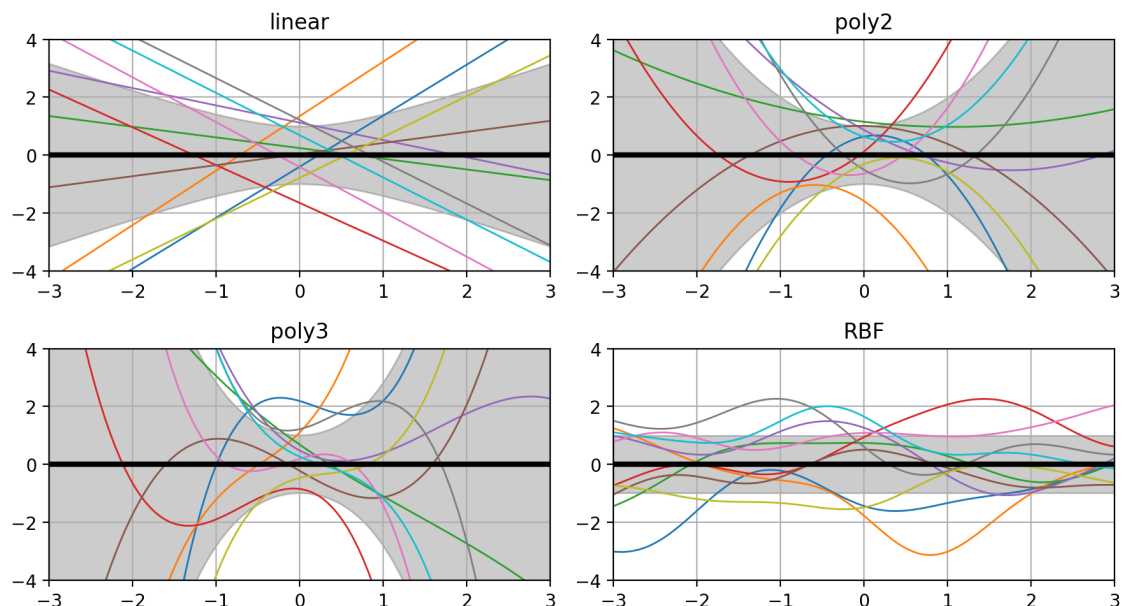
kernels = [DotProduct(), DotProduct()**2, DotProduct()**3, RBF()]
kernelnames = ['linear', 'poly2', 'poly3', 'RBF']

pfig = plt.figure(figsize=(9, 5))
for i,k in enumerate(kernels):
    # Specify Gaussian Process
    gp = gaussian_process.GaussianProcessRegressor(kernel=k)

    # Plot prior
    plt.subplot(2, 2, i+1)
    X_ = linspace(-3, 3, 100)
    y_mean, y_std = gp.predict(X_[:, newaxis], return_std=True)
    plt.plot(X_, y_mean, 'k', lw=3, zorder=9)
    plt.fill_between(X_, y_mean - y_std, y_mean + y_std,
                    alpha=0.2, color='k')
    y_samples = gp.sample_y(X_[:, newaxis], 10)
    plt.plot(X_, y_samples, lw=1)
    plt.xlim(-3, 3)
    plt.ylim(-4, 4)
    plt.title(kernelnames[i])
    plt.grid(True)
plt.tight_layout()
plt.close()
```

In [38]: pfig

Out[38]:



Gaussian Process Regression

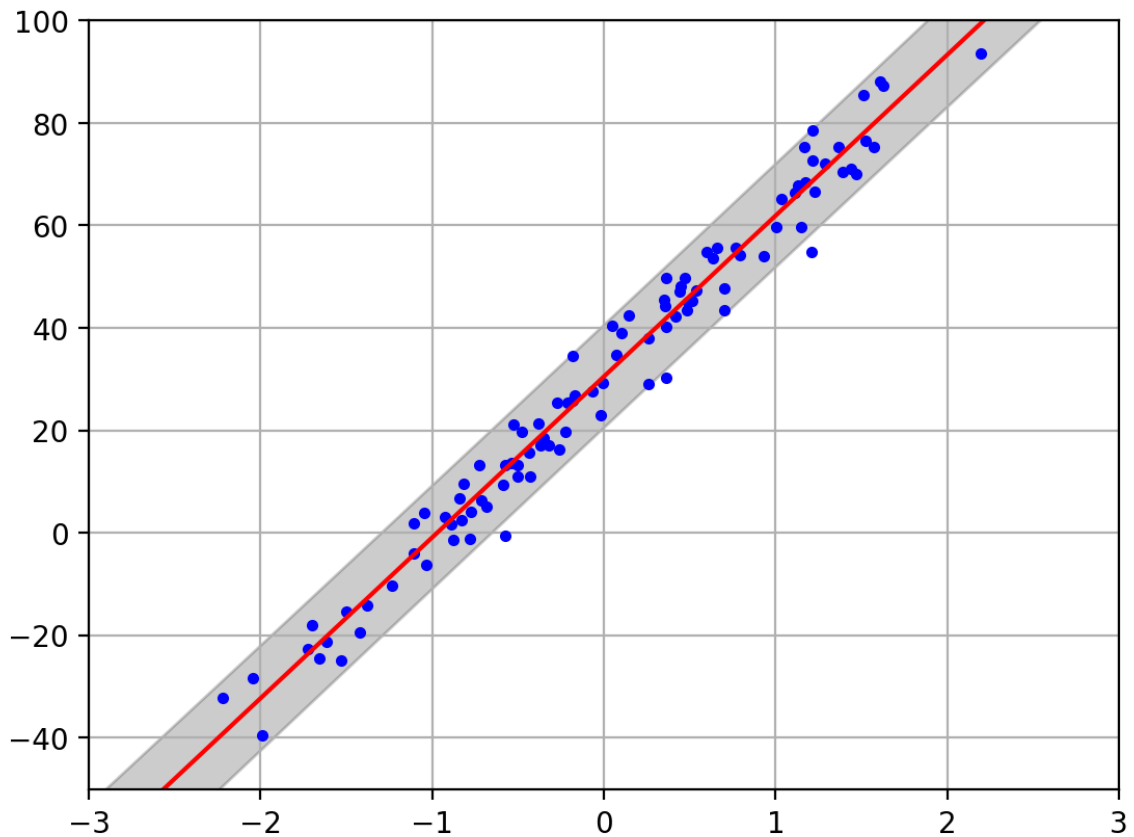
- Model framework
 - observation noise: $p(\mathbf{y}|\mathbf{f}) = \mathcal{N}(\mathbf{y}|\mathbf{f}, \sigma^2\mathbf{I})$
 - equivalent to mean-squared error loss
 - function prior: $\mathbf{f} \sim \mathcal{GP}(0, k(\mathbf{x}, \mathbf{x}'))$
- Training: given dataset $\{\mathbf{X}, \mathbf{y}\}$
 - compute the posterior distribution of the function values for the observed data:
 - $p(\mathbf{f}|\mathbf{X}, \mathbf{y}) = \frac{p(\mathbf{y}|\mathbf{f})p(\mathbf{f})}{p(\mathbf{y}|\mathbf{X})}$
- Inference: given a new point $\mathbf{x}_*$
 - $p(f_*|x_*, \mathbf{X}, \mathbf{y}) = \int p(f_*|x_*, \mathbf{f})p(\mathbf{f}|\mathbf{X}, \mathbf{y})d\mathbf{f}$
 - averages the predictions over all probable function values $\mathbf{f}$.

GP Prediction

- All distributions are Gaussian, so there is a closed-form solution.
- The predictive distribution is Gaussian:
 - $p(f_*|x_*, \mathbf{X}, \mathbf{y}) = \mathcal{N}(f_*|\mu_*, \sigma_*^2)$
 - mean of prediction: $\mu_* = \mathbf{k}_*^T(\mathbf{K} + \sigma^2\mathbf{I})^{-1}\mathbf{y}$
 - variance of predictions: $\sigma_*^2 = k_{**} - \mathbf{k}_*^T(\mathbf{K} + \sigma^2\mathbf{I})^{-1}\mathbf{k}_*$
 - where $k_{**} = k(\mathbf{x}_*, \mathbf{x}_*)$.
 - The uncertainty of the prediction is measured with its variance.
 - (higher values means more uncertain).
- GPR with a linear kernel is equivalent to Bayesian linear regression
 - `DotProduct()` - linear kernel: $k(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T\mathbf{x}' + \alpha_1$
 - `WhiteKernel()` - observation noise (σ^2): $k(\mathbf{x}, \mathbf{x}') = \sigma^2\delta(\mathbf{x} - \mathbf{x}')$
 - gray area shows 2 standard deviations around the mean (95% confidence region)

```
In [39]: lsaxbox = [-3,3,-2,4]
         laxbox = [-3, 3, -50, 100]
```

```
In [40]: from sklearn.gaussian_process.kernels import DotProduct, WhiteKernel
         k = DotProduct() + WhiteKernel()
         gpr = gaussian_process.GaussianProcessRegressor(kernel=k,
                                                         random_state=5489, normalize_y=True)
         # normalize_y: normalize Y to mean 0, var 1 (GPR will be better beh
         gpr.fit(linX, linY)
         plot_regr_trans_1d(gpr, laxbox, linX, linY)
```



```
In [41]: naxbox2 = [-5, 5, -2, 2]
```

Non-linear regression using kernels

- kernels functions allow non-linear regression
 - `DotProduct()` - linear: $k(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}' + \alpha_1$
 - `DotProduct()*2` - 2nd order polynomial:
 $k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + \alpha_1)^2$
 - `DotProduct()*3` - 3rd order polynomial:
 $k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + \alpha_1)^3$
 - `RBF()` - Radial-basis function: $k(\mathbf{x}, \mathbf{x}') = \exp(-\frac{1}{2\alpha_2^2} \|\mathbf{x} - \mathbf{x}'\|^2)$
- Applying the kernel trick to Bayesian linear regression will yield GPR

```
In [42]: from sklearn.gaussian_process.kernels import DotProduct, WhiteKernel
kernels = [ DotProduct() + WhiteKernel(),
            DotProduct()*2 + WhiteKernel(),
            DotProduct()*3 + WhiteKernel(),
            RBF() + WhiteKernel() ]

gpr = {}
for i,k in enumerate(kernels):
    gpr[i] = gaussian_process.GaussianProcessRegressor(kernel=k, ra
    gpr[i].fit(polyX, polyY)
```

```
In [43]: kernelnames = ['linear', 'poly2', 'poly3', 'RBF']
gprfig = plt.figure(figsize=(9,4))
```

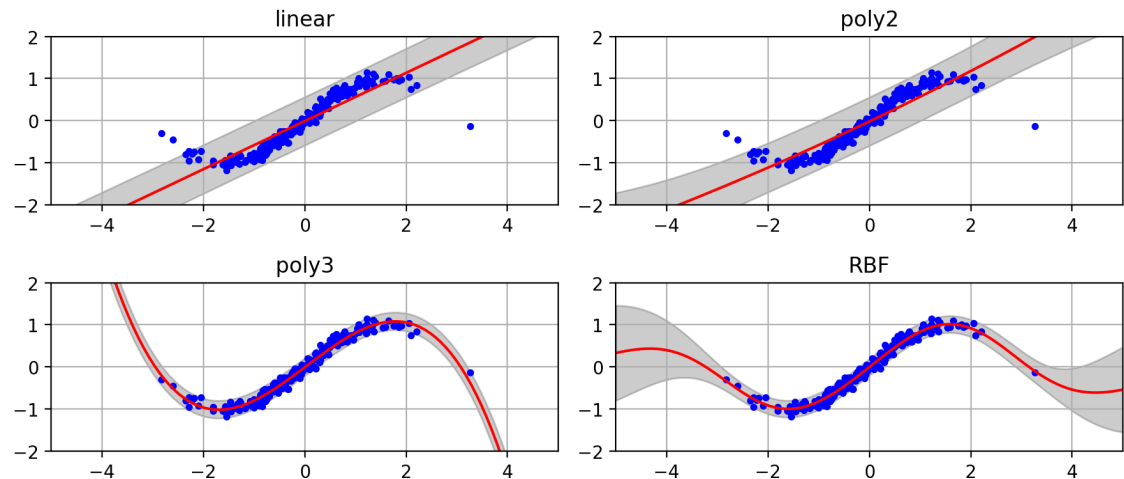
```

for i,k in enumerate(kernels):
    plt.subplot(2,2,i+1)
    plot_regr_trans_1d(gpr[i], naxbox2, polyX, polyY)
    plt.title(kernelnames[i])
plt.tight_layout()
plt.close()

```

In [44]: gprfig

Out[44]:



- kernels can be summed, multiplied, and exponentiated to make new kernels
 - e.g., `RBF() + DotProduct()*2 + WhiteKernel()`
 - regressed function is a sum of quadratic and RBF functions

```

In [45]: kernels = [ DotProduct() + RBF() + WhiteKernel(),
                    DotProduct()*2 + RBF() + WhiteKernel(),
                    DotProduct()*3 + RBF() + WhiteKernel(),
                    RBF(length_scale=0.01) + RBF() + WhiteKernel()]

gpr = {}
for i,k in enumerate(kernels):
    gpr[i] = gaussian_process.GaussianProcessRegressor(kernel=k, ra
    gpr[i].fit(polyX, polyY)

```

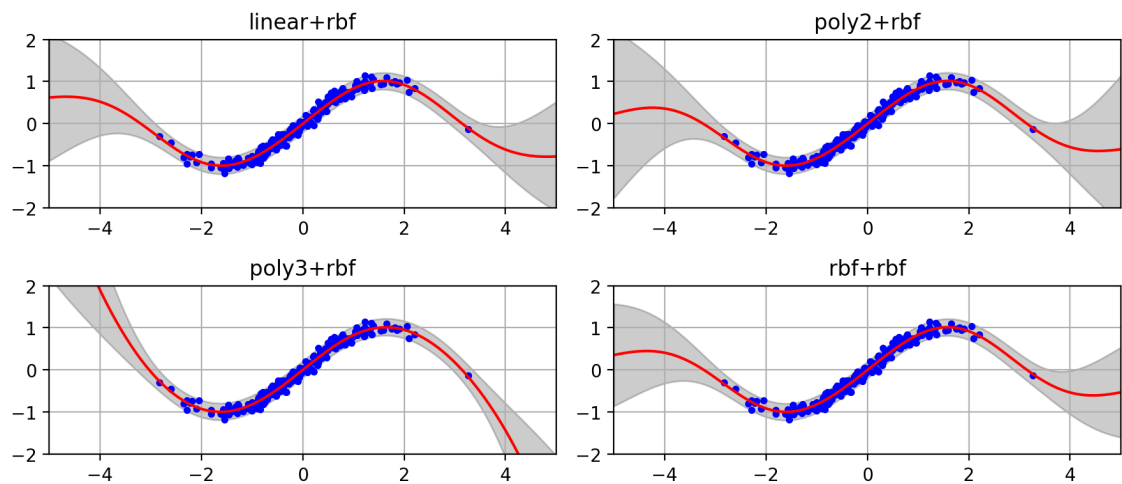
```

In [46]: kernelnames = ['linear+rbf', 'poly2+rbf', 'poly3+rbf', 'rbf+rbf']
gprfig = plt.figure(figsize=(9,4))
for i,k in enumerate(kernels):
    plt.subplot(2,2,i+1)
    plot_regr_trans_1d(gpr[i], naxbox2, polyX, polyY)
    plt.title(kernelnames[i])
plt.tight_layout()
plt.close()

```

In [47]: gprfig

Out[47]:



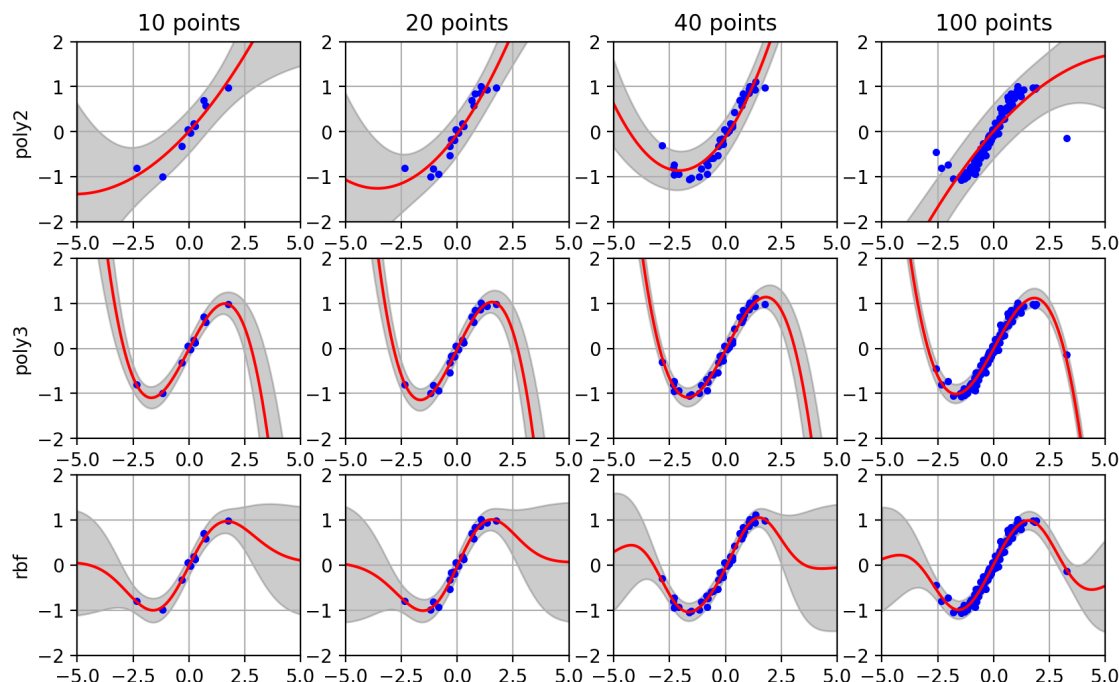
- As a Bayesian method, GPR handles lack of data well
 - the uncertainty (stddev of the prediction) increases when less data is available.

```
In [48]: kernels = [ DotProduct()2 + WhiteKernel(),
                    DotProduct()3 + WhiteKernel(),
                    RBF() + WhiteKernel() ]
kernelnames = ['poly2', 'poly3', 'rbf']

sfig = plt.figure(figsize=(10,6))
for j,k in enumerate(kernels):
    for i,s in enumerate([20,10,5,2]):
        gpr = gaussian_process.GaussianProcessRegressor(kernel=k, r
        gpr.fit(polyX[:,s], polyY[:,s])
        plt.subplot(3,4,i+1+j*4)
        plot_regr_trans_1d(gpr, naxbox2, polyX[:,s], polyY[:,s])
        if (i==0):
            plt.ylabel(kernelnames[j])
        if (j==0):
            plt.title("{} points".format(len(polyY[:,s])))
plt.close()
```

In [49]: sfig

Out[49]:



Estimation of Kernel Hyperparameters

- the hyperparameters of the kernel ($\alpha_1, \alpha_2, \sigma^2$, etc.) are estimated by maximizing the marginal likelihood:
 - marginal likelihood (aka model evidence)
 - $p(\mathbf{y}|\mathbf{X}) = \int p(\mathbf{y}|\mathbf{X}, \mathbf{f})p(\mathbf{f})d\mathbf{f}$
 - averages over all probable functions
 - Use iterative methods to maximize
 - $\alpha^* = \operatorname{argmax}_{\alpha} \log p(\mathbf{y}|\mathbf{X})$
- Advantages:
 - typically more efficient than grid-search when there are many kernel hyperparameters.
 - principled approach to model selection
- Disadvantage:
 - difficult optimization problem, possibly many local maximum

Intuition of MML

- Consider the space of datasets D .
 - A *constrained (simple)* model can only represent a few datasets.
 - the likelihood of data $p(D|M)$ for those datasets should be large, since it integrates to 1
 - A *flexible (complex)** model can represent many datasets.
 - the likelihood of data $p(D|M)$ for those datasets should be small.

- For a given D , by choosing the model with highest $p(D|M)$, we select the least complex model that fits the data.

No description has been provided for this image

Example on Housing Data

- try 2nd-order polynomial and RBF kernels.

```
In [50]: kernels = [DotProduct()*2 + WhiteKernel(),
                    RBF() + WhiteKernel()]
kernelnames = ['poly2', 'RBF']

gpr = [None]*2
for i,k in enumerate(kernels):
    gpr[i] = gaussian_process.GaussianProcessRegressor(
        kernel=k, random_state=5489,
        n_restarts_optimizer=5, normalize_y=True)
    gpr[i].fit(housingX, housingY)
```

/Users/jpm/miniconda3/envs/mldemo/lib/python3.13/site-packages/sklearn/gaussian_process/_gpr.py:663: ConvergenceWarning: lbfgs failed to converge after 18 iteration(s) (status=2):

ABNORMAL:

You might also want to scale the data as shown in:

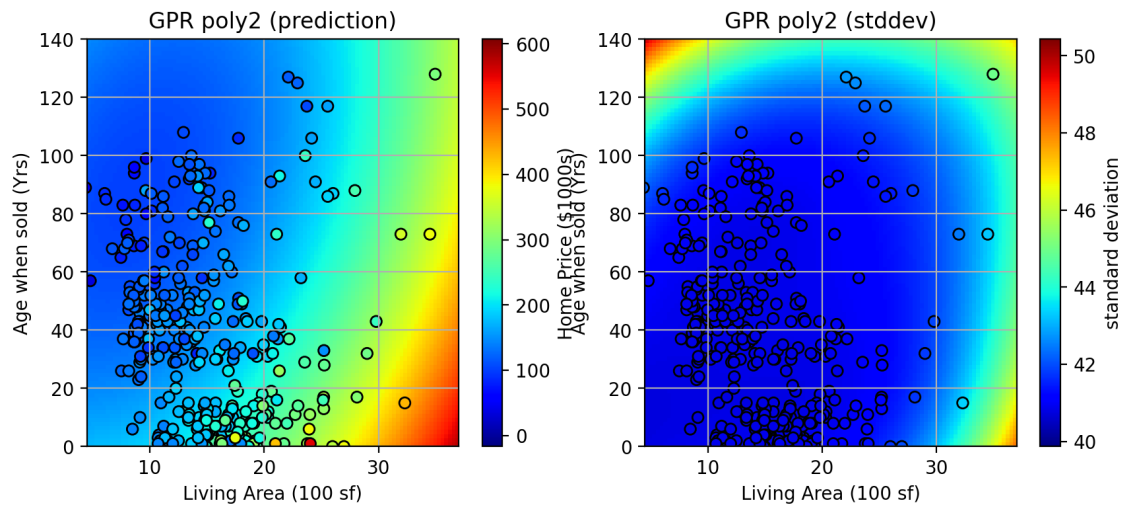
[https://scikit-learn.org/stable/modules/preprocessing.html#check_optimize_result\('lbfgs', opt_res\)](https://scikit-learn.org/stable/modules/preprocessing.html#check_optimize_result('lbfgs', opt_res))

```
In [51]: gprfig = [None]*2
for i,gg in enumerate(gpr):
    gprfig[i] = plt.figure(figsize=(10,4))
    plot_regr_trans_2d(gpr[i], housingaxbox2, housingX, housingY)
    plt.subplot(1,2,1)
    cbar = plt.colorbar()
    cbar.set_label(housingYname)
    plt.xlabel(housingXname[0])
    plt.ylabel(housingXname[1])
    plt.title('GPR {} (prediction)'.format(kernelnames[i]));
    plt.subplot(1,2,2)
    cbar = plt.colorbar()
    cbar.set_label('standard deviation')
    plt.xlabel(housingXname[0])
    plt.ylabel(housingXname[1])
    plt.title('GPR {} (stddev)'.format(kernelnames[i]));
    plt.close()
```

- 2nd order polynomial kernel
 - stddev of prediction shows when the model is not confident

```
In [52]: gprfig[0]
```

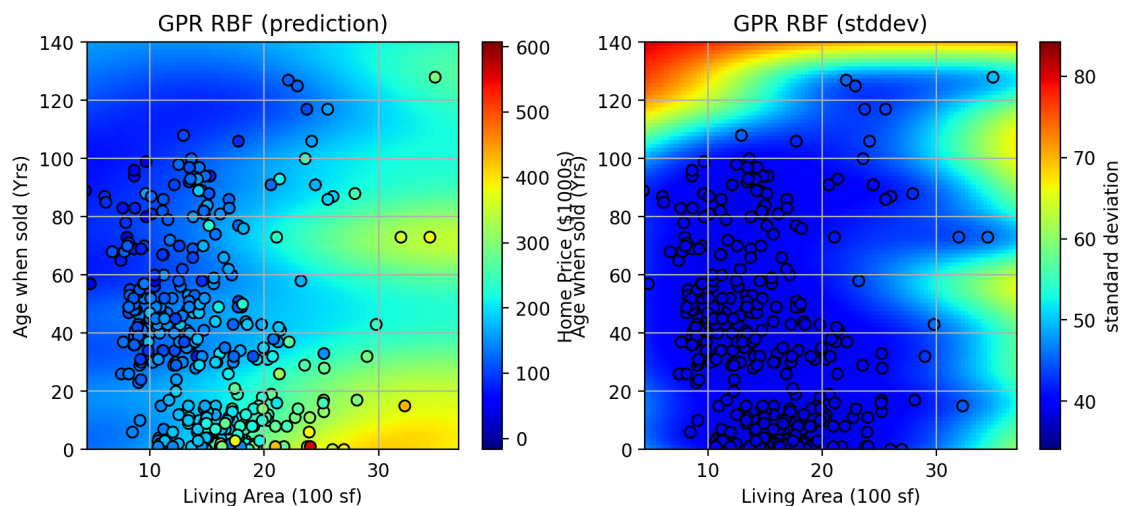
Out[52]:



- RBF kernel
 - stddev of prediction shows when the model is not confident
 - Since the RBF kernel has finite extent, it is not confident where it doesn't see data.

In [53]: `gprfig[1]`

Out[53]:



Support Vector Regression (SVR)

- Borrow ideas from classification
 - Suppose we form a "band" of width ϵ around the function:
 - if a point is inside, then it is "correctly" predicted
 - if a point is outside, then it is incorrectly predicted

```
In [54]: def plot_svr_1d(model, axbox, X, Y, label=False, showsv=False):
          xr = linspace(axbox[0], axbox[1], 100).reshape(100,1)
          Ypred = model.predict(xr)

          epsilon = model.epsilon
```



```

plt.plot(X, Y, 'b.', label='points')
plt.plot(xr, Ypred, 'r-', label='$f(x)$')
plt.plot(xr, Ypred+epsilon, 'r--', linewidth=1, label='$f(x)\\pm\\epsilon$')
plt.plot(xr, Ypred-epsilon, 'r--', linewidth=1)

if showsv:
    ypred = model.predict(xr)
    ss = model.support_ # get support vector indicies
    plt.plot(X[ss], Y[ss], 'r.', label='support vectors')

plt.axis(axbox); plt.grid(True)

if label:
    plt.legend(loc=0)

```

```

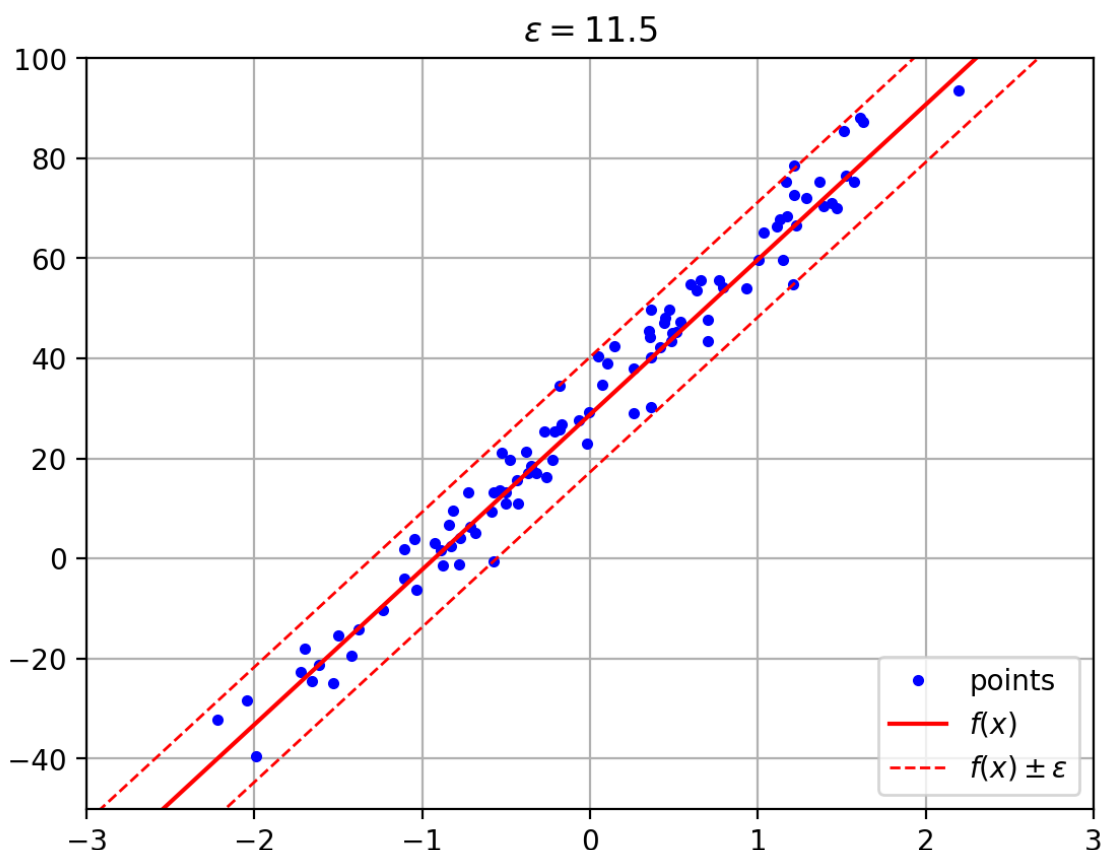
In [55]: epsilon = 11.5
svr = svm.SVR(C=1000, kernel='linear', epsilon=epsilon)
svr.fit(linX, linY)

axbox = [-3, 3, -50, 100]
svrfig = plt.figure()
plot_svr_1d(svr, laxbox, linX, linY)
plt.grid(True); plt.axis(laxbox)
plt.legend(loc='lower right')
plt.title('$\\epsilon=11.5$')
plt.close()

```

In [56]: svrfig

Out[56]:



- Allow some points to be outside the "tube".

- penalty of point outside tube is controlled by C parameter.
- SVR objective function:

$$\min_{\mathbf{w}, b} \sum_{i=1}^N |y_i - (\mathbf{w}^T \mathbf{x}_i + b)|_{\epsilon} + \frac{1}{C} \|\mathbf{w}\|^2$$

- find the least complex function with most points inside the tube.
- epsilon-insensitive error:
 - $|z|_{\epsilon} = \begin{cases} 0, & |z| \leq \epsilon \\ |z| - \epsilon, & |z| > \epsilon \end{cases}$
- Similar to SVM classifier, the points on the band will be the *support vectors* that define the function.

Different tube widths

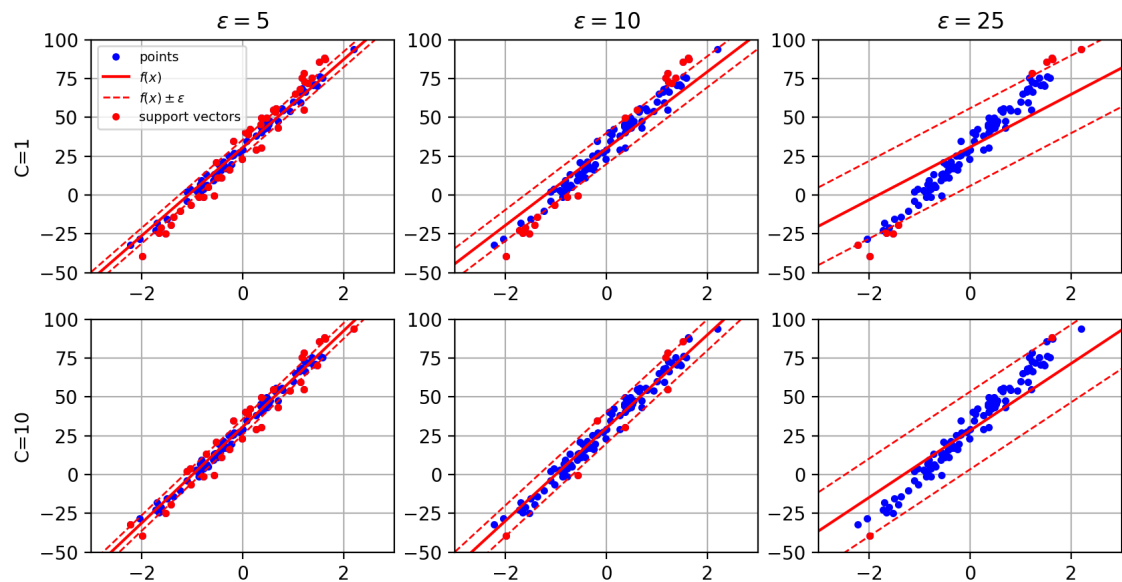
- The points on/outside the tube are the *support vectors*.

```
In [57]: svrfig = plt.figure(figsize=(10,5))
ind = 1
for C in [1,10]:
    for epsilon in [5,10,25]:
        svr = svm.SVR(C=C, kernel='linear', epsilon=epsilon)
        svr.fit(linX, linY)

        plt.subplot(2,3,ind)
        plot_svr_1d(svr, axbox, linX, linY, label=False, showsv=True)
        if (C==1):
            plt.title('$\\epsilon='+str(epsilon)+'$')
        if (epsilon==5):
            plt.ylabel('C='+str(C))
        if (ind==1):
            plt.legend(fontsize=7, loc='upper left')
        ind=ind+1
plt.close()
```

```
In [58]: svrfig
```

Out [58]:



Kernel SVR

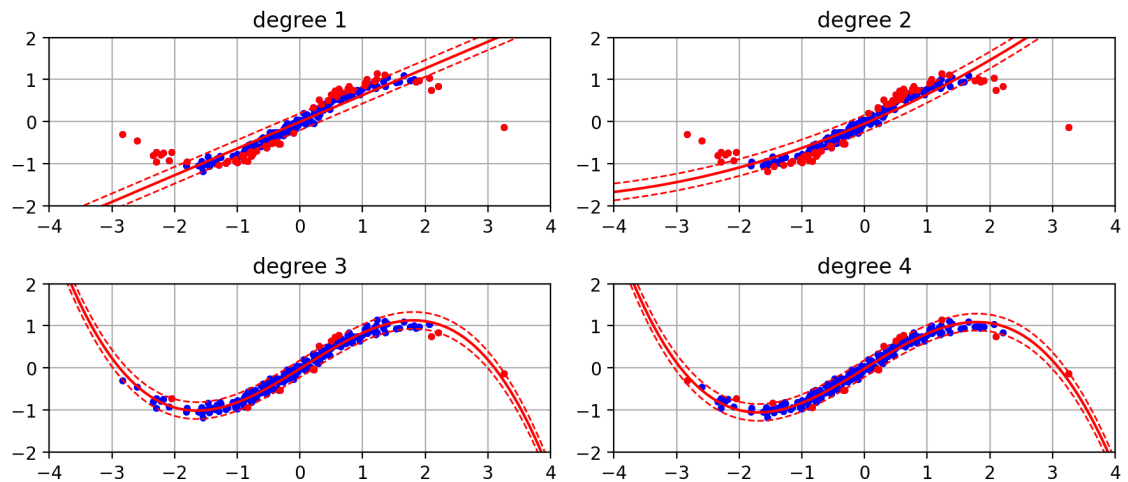
- Support vector regression can also be kernelized similar to SVM
 - turn linear regression to non-linear regression
- Polynomial Kernel:

```
In [59]: epsilon = 0.2
svr = {}
for d in [1,2,3,4]:
    # fit the parameters (poly SVR)
    svr[d] = svm.SVR(C=1000, kernel='poly', coef0=0.1, degree=d, epsilon=epsilon)
    svr[d].fit(polyX, polyY)
```

```
In [60]: svrfig=plt.figure(figsize=(9,4))
for d in [1,2,3,4]:
    plt.subplot(2,2,d)
    plot_svr_1d(svr[d], naxbox, polyX, polyY, showsv=True)
    plt.title("degree " + str(d))
plt.tight_layout()
plt.close()
```

In [61]: svrfig

Out[61]:

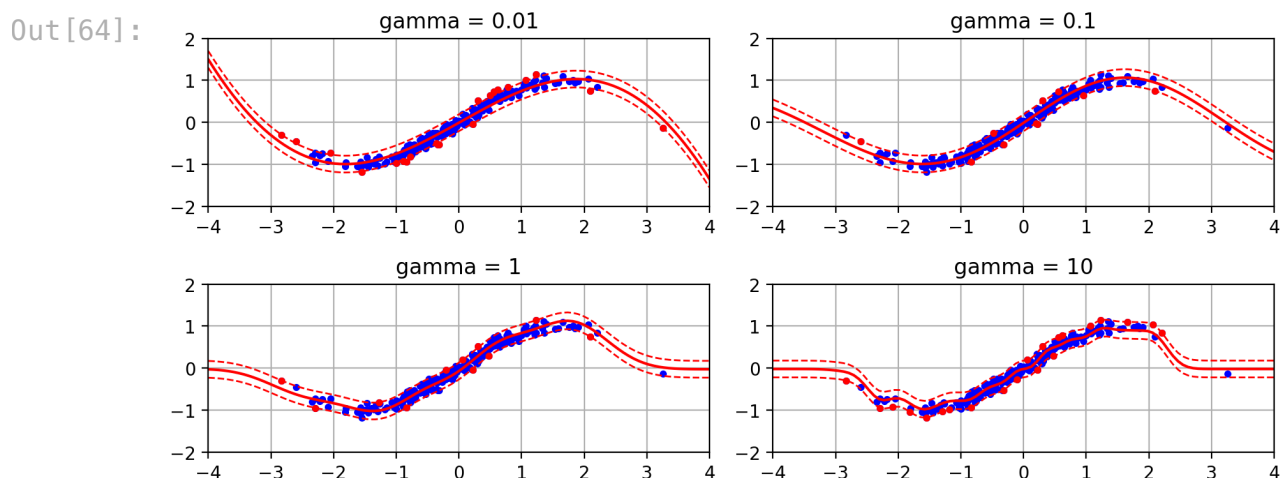


SVR with RBF kernel

```
In [62]: epsilon = 0.2
svr = {}
for i,g in enumerate([0.01, 0.1, 1, 10]):
    # fit the parameters: SVR with RBF
    svr[i] = svm.SVR(C=1000, kernel='rbf', gamma=g, epsilon=epsilon)
    svr[i].fit(polyX, polyY)
```

```
In [63]: svrfig = plt.figure(figsize=(9,4))
for i,g in enumerate([0.01, 0.1, 1, 10]):
    plt.subplot(2,2,i+1)
    plot_svr_1d(svr[i], naxbox, polyX, polyY, showsv=True)
    plt.title("gamma = " + str(g))
plt.tight_layout()
plt.close()
```

In [64]: svrfig



Housing Data

- Cross-validation to select 3 parameters

▪ C, γ, ϵ

```
In [65]: # parameters for cross-validation
paramgrid = {'C':      logspace(-3,3,10),
             'gamma':  logspace(-3,3,10),
             'epsilon': logspace(-2,2,10)}

# do cross-validation
svrcv = model_selection.GridSearchCV(
    svm.SVR(kernel='rbf'), # estimator
    paramgrid,             # parameters to try
    scoring='neg_mean_squared_error', # score function
    cv=5,
    n_jobs=-1, verbose=1) # show progress
svrcv.fit(housingX, housingY)

print(svrcv.best_score_)
print(svrcv.best_params_)
```

Fitting 5 folds for each of 1000 candidates, totalling 5000 fits
-1558.6869614323266
{'C': np.float64(215.44346900318823), 'epsilon': np.float64(12.915496650148826), 'gamma': np.float64(0.004641588833612777)}

```
In [66]: (avgscores, pnames, bestind) = extract_grid_scores(svrcv, paramgrid)

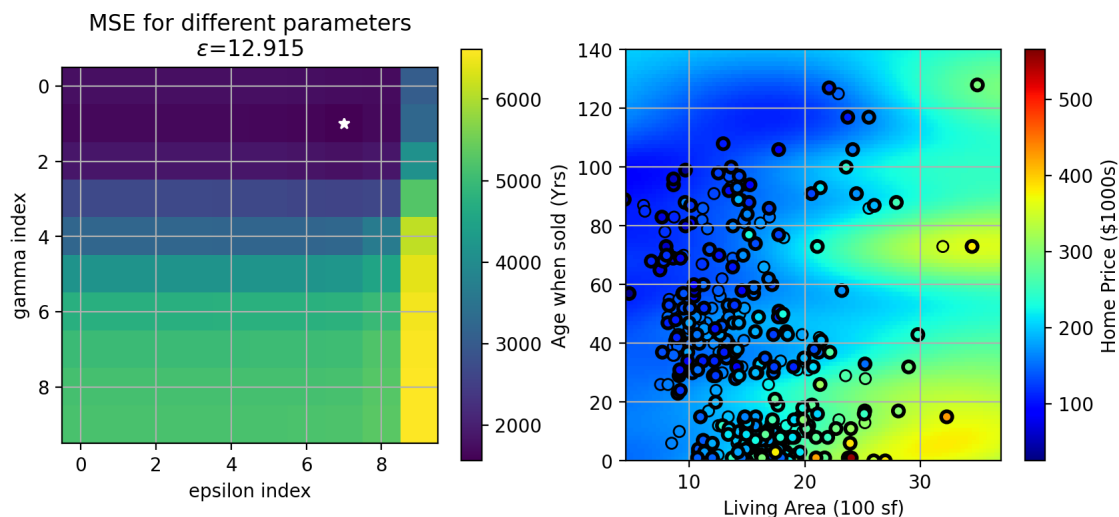
kfig = plt.figure(figsize=(10,4))
# show score map
plt.subplot(1,2,1)
plt.imshow(squeeze(-avgscores[bestind[0],:,:]), interpolation='nearest')
plt.plot(bestind[2], bestind[1], '*w')
plt.ylabel(pnames[1] + ' index'); plt.xlabel(pnames[2] + ' index')
plt.grid(True)
plt.title('MSE for different parameters\n$\epsilon$={:.3f}'.format(
plt.colorbar(); plt.axis('image')

# show regression function
vmin = housingY.min()-10
vmax = housingY.max()+10
plt.subplot(1,2,2)
plot_regr_trans_2d(svrcv, housingaxbox2, housingX, housingY)
ss = svrcv.best_estimator_.support_
plt.scatter(housingX[ss,0], housingX[ss,1], c=housingY[ss], vmin=vmin,
cbar = plt.colorbar()
cbar.set_label(housingYname)
plt.xlabel(housingXname[0]); plt.ylabel(housingXname[1]);
plt.close()
```

```
<>:10: SyntaxWarning: invalid escape sequence '\e'
<>:10: SyntaxWarning: invalid escape sequence '\e'
/var/folders/v0/s4761j_n3z7_f651dfw62m5w0000gn/T/ipykernel_8114/2723
365022.py:10: SyntaxWarning: invalid escape sequence '\e'
    plt.title('MSE for different parameters\n$\epsilon$={:.3f}'.forma
t(svrcv.best_params_['epsilon']))
```

```
In [67]: kfig
```

Out[67]:



Model Ensemble

- *Why trust only one model (expert)?*
 - In real life, we may consult several experts, or go with the "wisdom of the crowd"
 - In machine learning, *why trust only one regression model?*
- Ensemble methods aim to combine multiple regression model together to form a better algorithm.
- Examples:
 - **bagging** - training multiple regression models from random selection of training data
 - **boosting** - training multiple regression models, each focusing on errors made by previous model.

Decision Tree (Regression Tree)

- Simple "Rule-based" regression
 - At each node, move down the tree based on that node's criteria.
 - leaf node contains the prediction
- **Advantage:** can create complex conjunction of rules
- **Disadvantage:** easy to overfit by itself

Random Forest Regression

- Use **bagging** to make an ensemble of Decision Tree Regressor

- for each *Decision Tree Regressor*
 - create a new training set by randomly sampling from the training set
 - for each split in a tree, select a random subset of features to use
 - Leaf node of tree contains the predicted value.
- for a test sample, the prediction is the average value of the target variable in the leaf node that the data point lands in.

No description has been provided for this image

- **Advantages**
 - can do feature selection.
 - good generalization.
 - fast.
- **Disadvantages**
 - can be sensitive to outliers

```
In [68]: def plot_rf_1d(model, axbox, X, Y, label=False, showdt=False, numx=
xr = linspace(axbox[0], axbox[1], numx).reshape(numx,1)
Ypred = model.predict(xr)

if showdt:
    myalpha=0.25
else:
    myalpha=1.0
plt.plot(X, Y, 'b.', label='points', alpha=myalpha)

if showdt:
    for i in model.estimators_:
        yy = i.predict(xr)
        plt.plot(xr, yy, 'm-')

plt.plot(xr, Ypred, 'r-', label='forest', lw=2)
plt.axis(axbox); plt.grid(True)

if label:
    plt.legend(loc=0)
```

Example

- Four decision trees
 - the regressed function has "steps" because of the decision tree has a constant prediction for ranges of feature values.

```
In [69]: rf = ensemble.RandomForestRegressor(n_estimators=4, random_state=44)
rf.fit(polyX, polyY)
```

Out[69]:

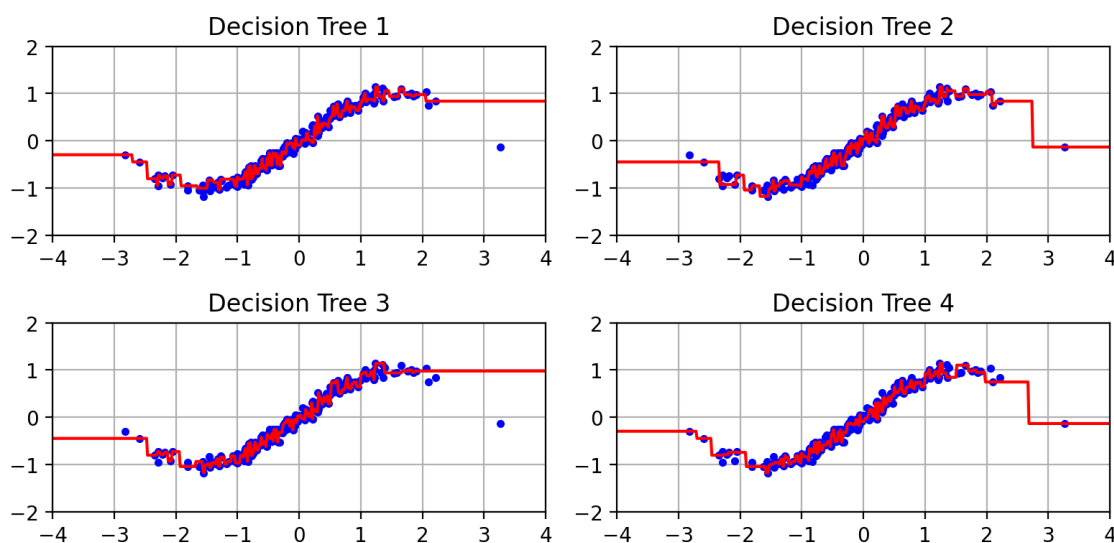
▼ RandomForestRegressor ⓘ ?

► Parameters

```
In [70]: rffig = plt.figure(figsize=(8,4))
for i in range(4):
    plt.subplot(2,2,i+1)
    plot_regr_trans_1d(rf.estimators_[i], naxbox, polyX, polyY, num
    plt.title('Decision Tree ' + str(i+1))
plt.tight_layout()
plt.close()
```

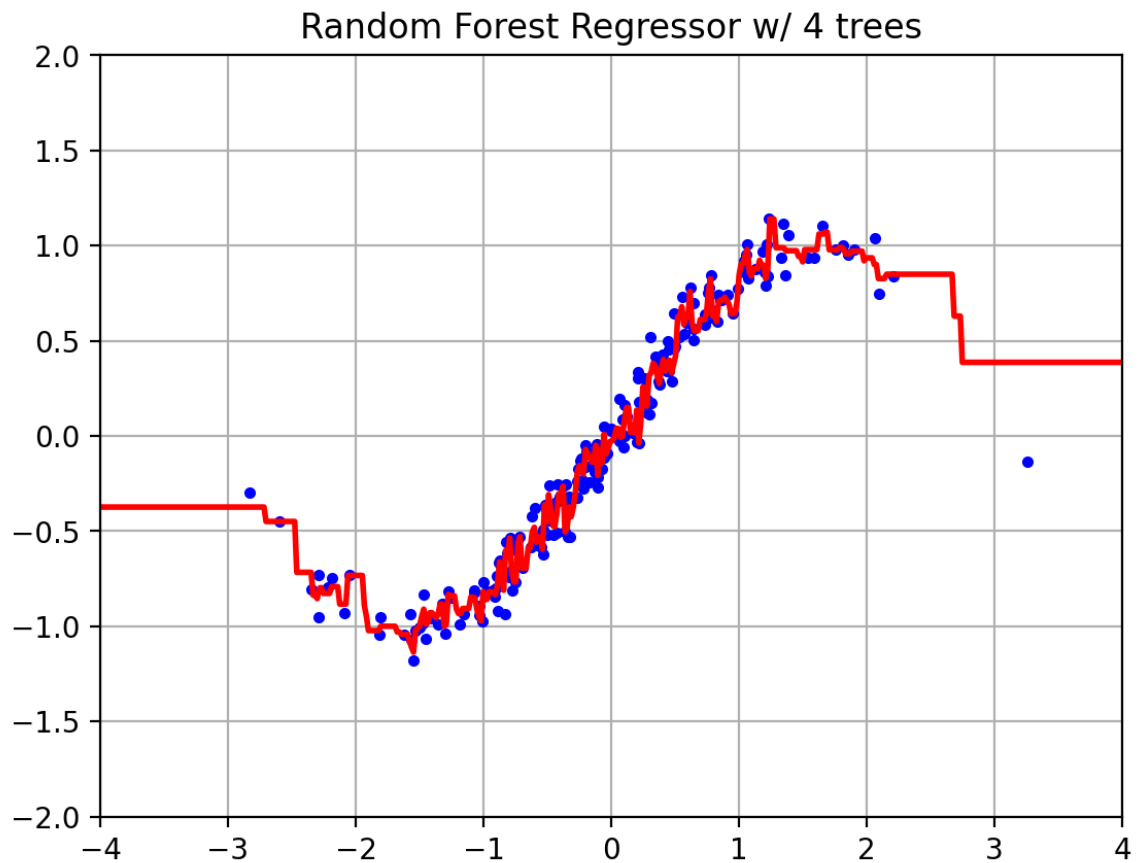
In [71]: rffig

Out[71]:



```
In [72]: # the aggregated function
plt.figure()
plot_rf_1d(rf, naxbox, polyX, polyY, numx=500)
plt.title('Random Forest Regressor w/ 4 trees')
```

Out[72]: Text(0.5, 1.0, 'Random Forest Regressor w/ 4 trees')



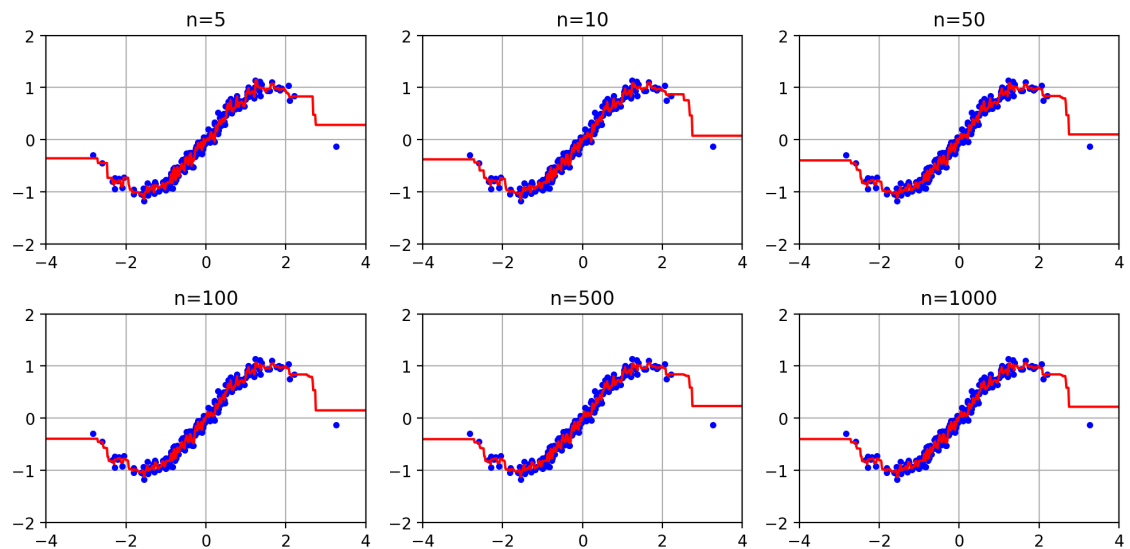
- Using more trees...

```
In [73]: rf = {}  
for i,n in enumerate([5, 10, 50, 100, 500, 1000]):  
    rf[i] = ensemble.RandomForestRegressor(n_estimators=n, random_s  
    rf[i].fit(polyX, polyY)
```

```
In [74]: rffig = plt.figure(figsize=(10,5))  
for i,n in enumerate([5, 10, 50, 100, 500, 1000]):  
    plt.subplot(2,3,i+1)  
    plot_regr_trans_1d(rf[i], naxbox, polyX, polyY, numx=500)  
    plt.title('n='+str(n))  
plt.tight_layout()  
plt.close()
```

```
In [75]: rffig
```

Out[75]:



Housing data

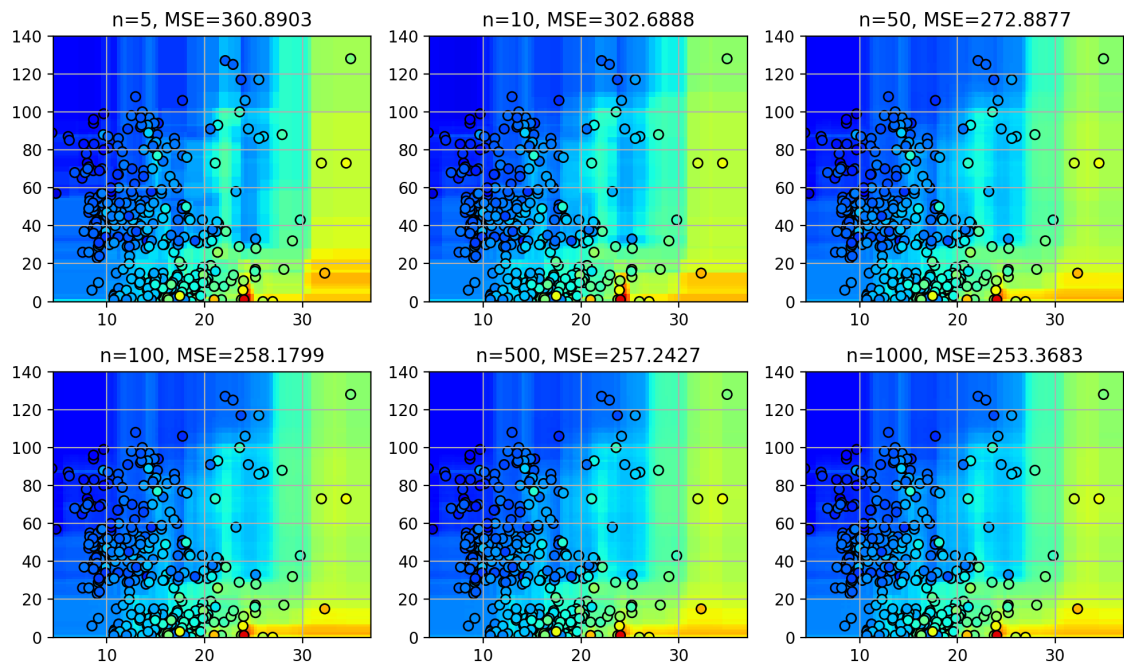
- The regressed function looks "blocky"
 - looks more reasonable for areas without any data

```
In [76]: rf = {}; MSE = {}  
for i,n in enumerate([5, 10, 50, 100, 500, 1000]):  
    rf[i] = ensemble.RandomForestRegressor(n_estimators=n, random_s  
    rf[i].fit(housingX, housingY)  
    MSE[i] = metrics.mean_squared_error(housingY, rf[i].predict(hou
```

```
In [77]: rffig = plt.figure(figsize=(10,6))  
for i,n in enumerate([5, 10, 50, 100, 500, 1000]):  
    plt.subplot(2,3,i+1)  
    plot_regr_trans_2d(rf[i], housingaxbox2, housingX, housingY)  
    plt.title('n={}, MSE={:.4f}'.format(n, MSE[i]))  
plt.tight_layout()  
plt.close()
```

```
In [78]: rffig
```

Out[78]:



- plot of MSE versus number of trees

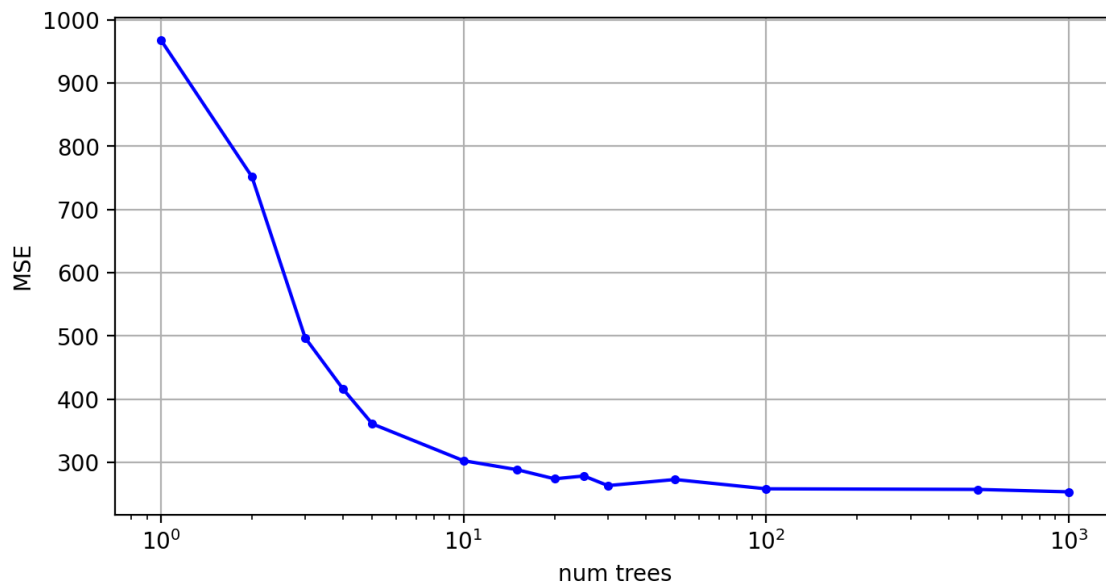
```
In [79]: ns = [1, 2, 3, 4, 5, 10, 15, 20, 25, 30, 50, 100, 500, 1000]
mSES = []
for n in ns:
    rf = ensemble.RandomForestRegressor(n_estimators=n, random_state=42)
    rf.fit(housingX, housingY)

    MSE = metrics.mean_squared_error(housingY, rf.predict(housingX))
    mSES.append(MSE)

mfig = plt.figure(figsize=(8,4))
plt.semilogx(ns, mSES, 'b.-')
plt.xlabel('num trees')
plt.ylabel('MSE')
plt.grid(True)
plt.close()
```

In [80]: mfig

Out[80]:



- Use cross-validation to select the tree depth

```
In [81]: # parameters for cross-validation
paramgrid = {'max_depth': array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15])
            }

# do cross-validation
rfcv = model_selection.GridSearchCV(
    ensemble.RandomForestRegressor(n_estimators=100, random_state=4,
    paramgrid,                                # parameters to try
    scoring='neg_mean_squared_error',          # score function
    cv=5,
    n_jobs=-1, verbose=True
)
rfcv.fit(housingX, housingY)

print(rfcv.best_score_)
print(rfcv.best_params_)
```

Fitting 5 folds for each of 11 candidates, totalling 55 fits
-1717.1838417959073
{'max_depth': np.int64(5)}

```
In [82]: (avgscores, pnames, bestind) = extract_grid_scores(rfcv, paramgrid)

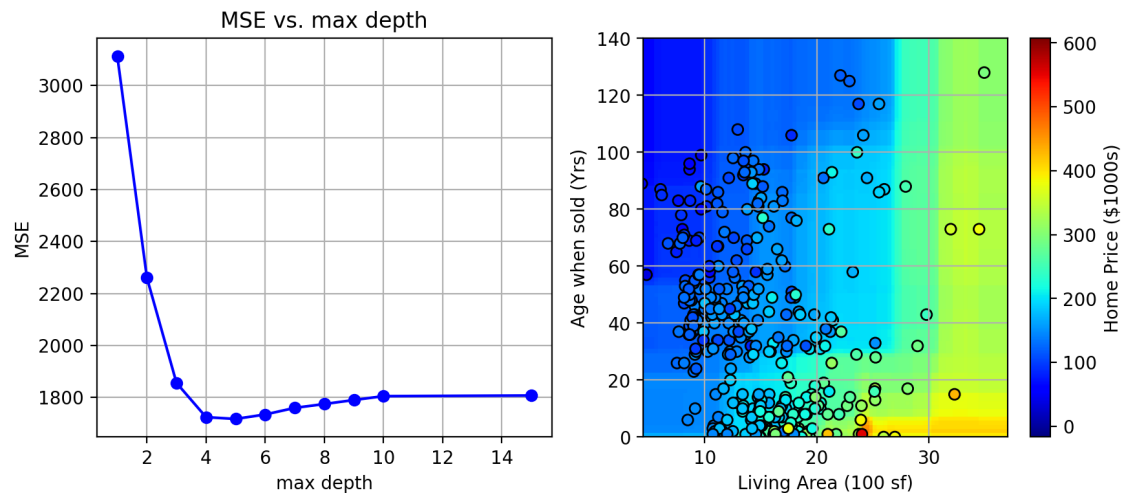
rffig = plt.figure(figsize=(10,4))
# show scores
plt.subplot(1,2,1)
plt.plot(paramgrid['max_depth'], -avgscores, 'bo-')
plt.xlabel('max depth'); plt.ylabel('MSE')
plt.title('MSE vs. max depth')
plt.grid(True)

# show regression function
plt.subplot(1,2,2)
plot_regr_trans_2d(rfcv, housingaxbox2, housingX, housingY)
cbar = plt.colorbar()
cbar.set_label(housingYname)
```

```
plt.xlabel(housingXname[0]); plt.ylabel(housingXname[1]);
plt.close()
```

In [83]: rffig

Out[83]:



Boosting

- Boosting is an ensemble learning method that combines a set of less accurate models (called "weak learners") to create a single, highly accurate model (a "strong learner")
- Boosting algorithms build models sequentially where each new model in the sequence is trained to correct the errors made by its predecessors (higher weights on samples with larger losses by predecessors).

XGBoost Regression

- variant of boosting
 - each iteration fits the residual (gap) between the current predictions and the groundtruth
 - the residual is computed as the gradient of the loss function.
- It's a gradient descent algorithm
 - in each iteration, the weak learner fits the gradient of the loss
 - $h_t(\mathbf{x}) \approx \frac{dL}{d\mathbf{f}}$
- and adds it to the function:
 - $f_t(\mathbf{x}) = f_{t-1}(\mathbf{x}) - \alpha_t h_t(\mathbf{x}) \approx f_{t-1}(\mathbf{x}) - \alpha_t \frac{dL}{d\mathbf{f}_{t-1}}$
- Generalizes boosting to other loss functions.
- Regression loss:
 - `reg:squarederror` should be used when minimizing squared loss (Gaussian noise assumption).
 - `reg:gamma` should be used when regression with Gamma noise

assumption (non-negative values).

- others...see [documentation](#).

```
In [84]: # setup dictionary of distributions for each parameter
paramsampler = {
    "colsample_bytree": stats.uniform(0.7, 0.3), # default=1
    "gamma": stats.uniform(0, 0.5), # default=0
    "max_depth": stats.randint(2, 6), # default=6
    "subsample": stats.uniform(0.6, 0.4), # default=1
    "learning_rate": stats.uniform(.001,1), # default=1 (could be 0.01)
    "n_estimators": stats.randint(10, 1000),
}

# the XGB regressor using squared error loss
xr = xgb.XGBRegressor(objective="reg:squarederror", random_state=44)

# cross-validation via random search
# n_iter = number of parameter combinations to try
xgbrcv = model_selection.RandomizedSearchCV(xr, param_distributions=paramsampler,
                                             scoring='neg_mean_squared_error', # score
                                             random_state=4487, n_iter=200, cv=5,
                                             verbose=1, n_jobs=6)

xgbrcv.fit(housingX, housingY)
print("best params:", xgbrcv.best_params_)
```

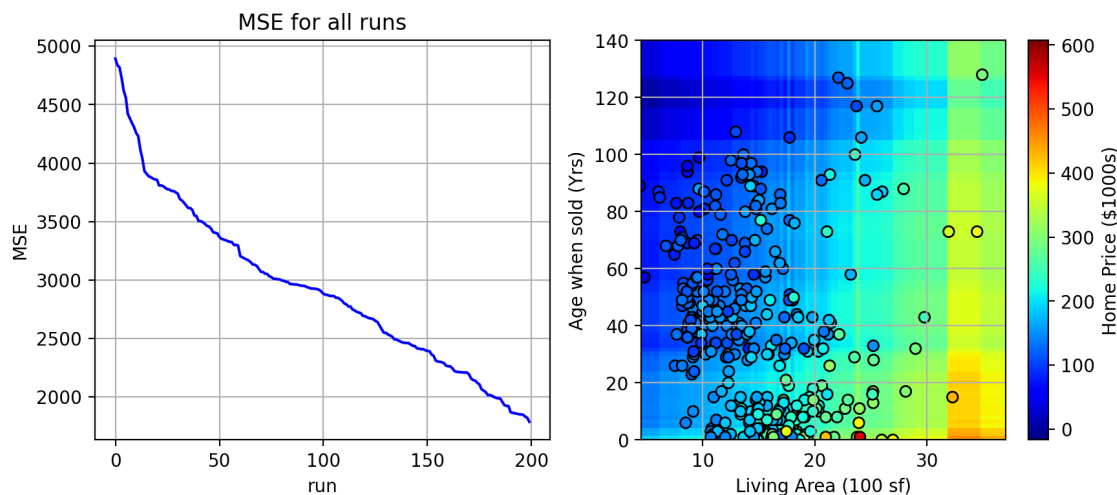
Fitting 5 folds for each of 200 candidates, totalling 1000 fits
best params: {'colsample_bytree': np.float64(0.8168908224647844), 'gamma': np.float64(0.35398611989014306), 'learning_rate': np.float64(0.06248414027218807), 'max_depth': 2, 'n_estimators': 274, 'subsample': np.float64(0.6240108914892146)}

```
In [85]: xfig = plt.figure(figsize=(10,4))
# show scores
plt.subplot(1,2,1)
plt.plot(range(200), -sort(xgbrcv.cv_results_['mean_test_score']),
plt.xlabel('run'); plt.ylabel('MSE')
plt.title('MSE for all runs')
plt.grid(True)

# show regression function
plt.subplot(1,2,2)
plot_regr_trans_2d(xgbrcv, housingaxbox2, housingX, housingY)
cbar = plt.colorbar()
cbar.set_label(housingYname)
plt.xlabel(housingXname[0]); plt.ylabel(housingXname[1]);
plt.close()
```

```
In [86]: xfig
```

Out[86]:



Regression Summary

- **Goal:** predict output $y \in \mathbb{R}$ from input $\mathbf{x} \in \mathbb{R}^d$.
 - i.e., learn the function $y = f(\mathbf{x})$.

Name	Function	Training	Advantages	Disadvantages
Ordinary Least Squares	linear	minimize square error between observation and predicted output.	- closed-form solution.	- sensitive to outliers and overfitting.
ridge regression	linear	minimize squared error with $\ w\ ^2$ (L2-norm) regularization term.	- closed-form solution; - shrinkage to prevent overfitting.	- sensitive to outliers.
LASSO	linear	minimize squared error with $\sum_{j=1}^d w_j $ (L1-norm) regularization term.	- feature selection (by forcing weights to 0)	- sensitive to outliers.
OMP	linear	minimize squared error with constraint on number of non-zero weights (L0-norm).	- feature selection	- difficult optimization problem, sensitive to outliers.
RANSAC	same as the base model	randomly sample subset of training data and fit model; keep model with most inliers.	- ignores outliers.	- requires enough iterations to find good consensus set.
kernel ridge regression	non-linear (kernel function)	apply "kernel trick" to ridge regression.	- non-linear regression. - Closed-form solution.	- requires calculating kernel matrix $O(N^2)$. - cross-validation to select hyperparameters.
			- non-linear regression - Closed-form	

Gaussian process regression	non-linear (kernel function)	- compute posterior distribution - estimate hyperparameters via MML.	solution. - works well with small datasets - hyperparameter estimation	- requires calculating kernel matrix $O(N^2)$.
kernel support vector regression	non-linear (kernel function)	minimize squared error, insensitive to epsilon-error.	- non-linear regression. - faster predictions than kernel ridge regression.	- requires calculating kernel matrix $O(N^2)$. - iterative solution (slow). - cross-validation to select hyperparameters.
random forest regression	non-linear (ensemble)	aggregate predictions from decision trees.	- non-linear regression. - fast predictions.	- predicts step-wise function. - cannot learn a completely smooth function.
XGBoost regression	non-linear (ensemble)	aggregaate predictions from decision trees.	- non-linear regression. - fast predictions.	- predicts step-wise function. - cannot learn a completely smooth function.

Other Things

- *Feature normalization*
 - feature normalization is typically required for regression methods with regularization.
 - makes ordering of weights more interpretable (LASSO, RR).
- *Output transformations*
 - sometimes the output values y have a large dynamic range (e.g., 10^{-1} to 10^5).
 - large output values will have large error, which will dominate the training error.
 - in this case, it is better to transform the output values using the logarithm function.
 - $\hat{y} = \log_{10}(y)$, for example, see the tutorial.
 - Gaussian process regression assumes y values are zero-mean and unit variance.
 - need to normalize to make it well behaved.